

Introduction to Algorithm

기초 알고리즘

다이나믹 프로그래밍 개요 (1)



기존 풀이 전략의 한계

재귀 호출

완전 탐색

탐욕적 기법



기존 풀이 전략의 한계

재귀 호출

완전 탐색

탐욕적 기법

함수가 자기 자신을 반복적으로 호출하는 재귀 호출



기존 풀이 전략의 한계

재귀 호출

완전 탐색

탐욕적 기법

주어진 모든 경우의 수를 구석구석 탐색하는 **완전 탐색**



기존 풀이 전략의 한계

재귀 호출

완전 탐색

탐욕적 기법

작은 순간마다 최선의 이득이 되도록 탐욕적으로 선택해
최적의 결과를 만들어내는 탐욕적 기법



기존 풀이 전략의 한계

재귀 호출

완전 탐색

탐욕적 기법

기존 풀이 전략의 한계를 하나씩 살펴보고
더 좋은 방법을 고민해보자

재귀 호출의 한계



재귀 호출의 한계 (1) – 시간과 메모리

```
def adder(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder(n - 1)
```

재귀 호출은 반복을 수행하는 하나의 방법



재귀 호출의 한계 (1) – 시간과 메모리

```
def adder(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder(n - 1)
```

```
def adderForLoop(n):  
    result = 0  
    for i in range(1, n + 1):  
        result += i  
    return result
```

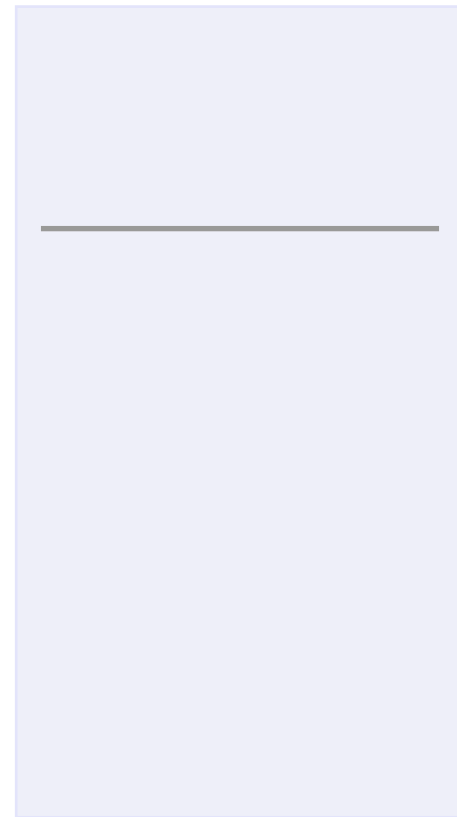
마치 for 반복문처럼

과연 재귀 호출은 for 반복문보다 좋은 방법일까?



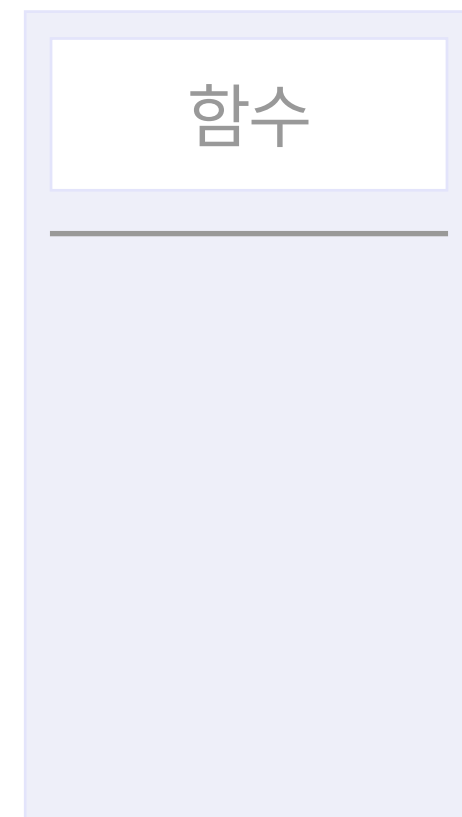
재귀 호출의 한계 (1) – 시간과 메모리

스택(Stack)



재귀 호출을 포함한 모든 함수는 호출되는 즉시

스택(Stack)



호출 스택에 스택 프레임이 생성되어
해당 함수의 실행 문맥이 저장됨

스택(Stack)



호출 스택에 스택 프레임이 생성되어
해당 함수의 실행 문맥이 저장됨

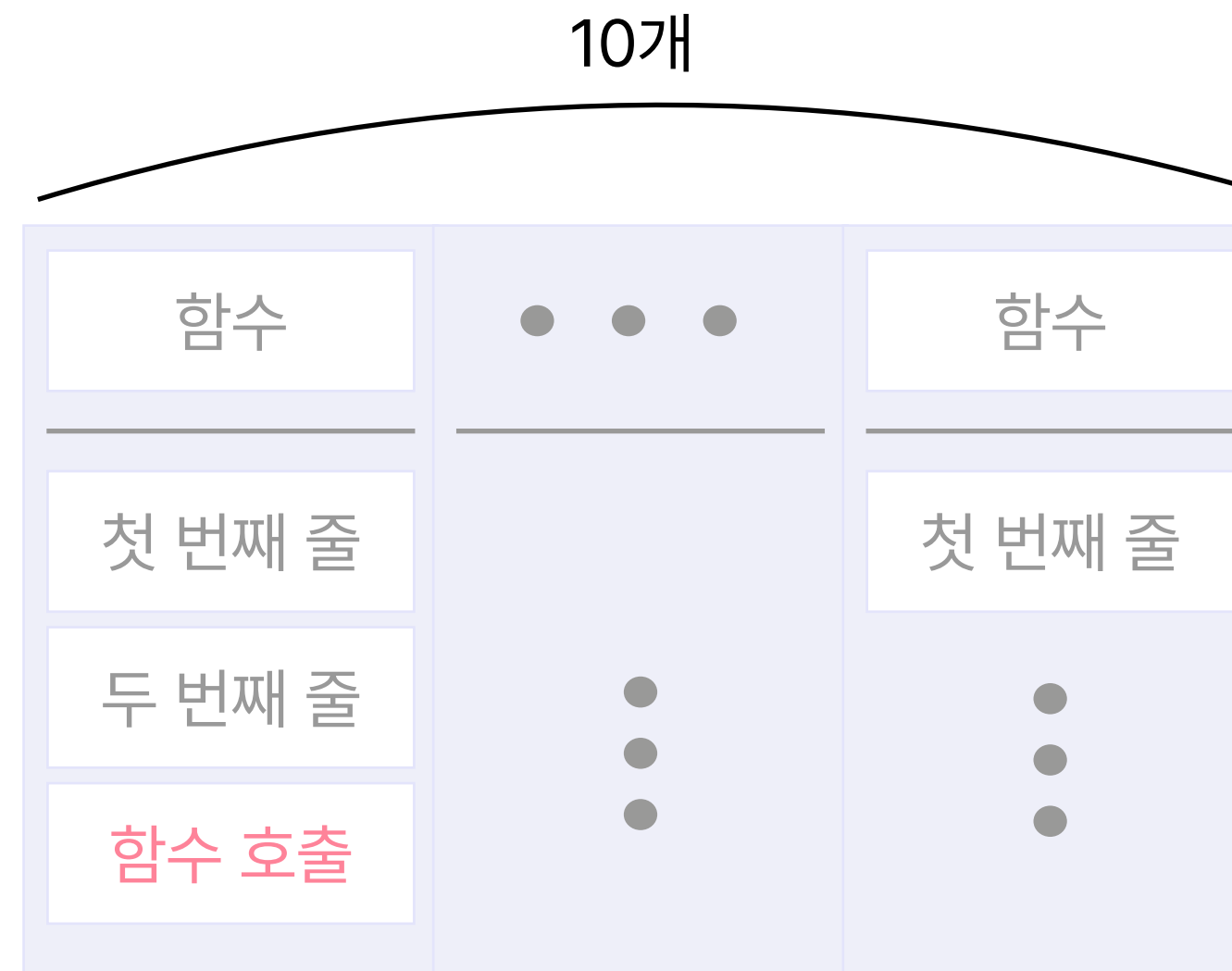


스택(Stack)



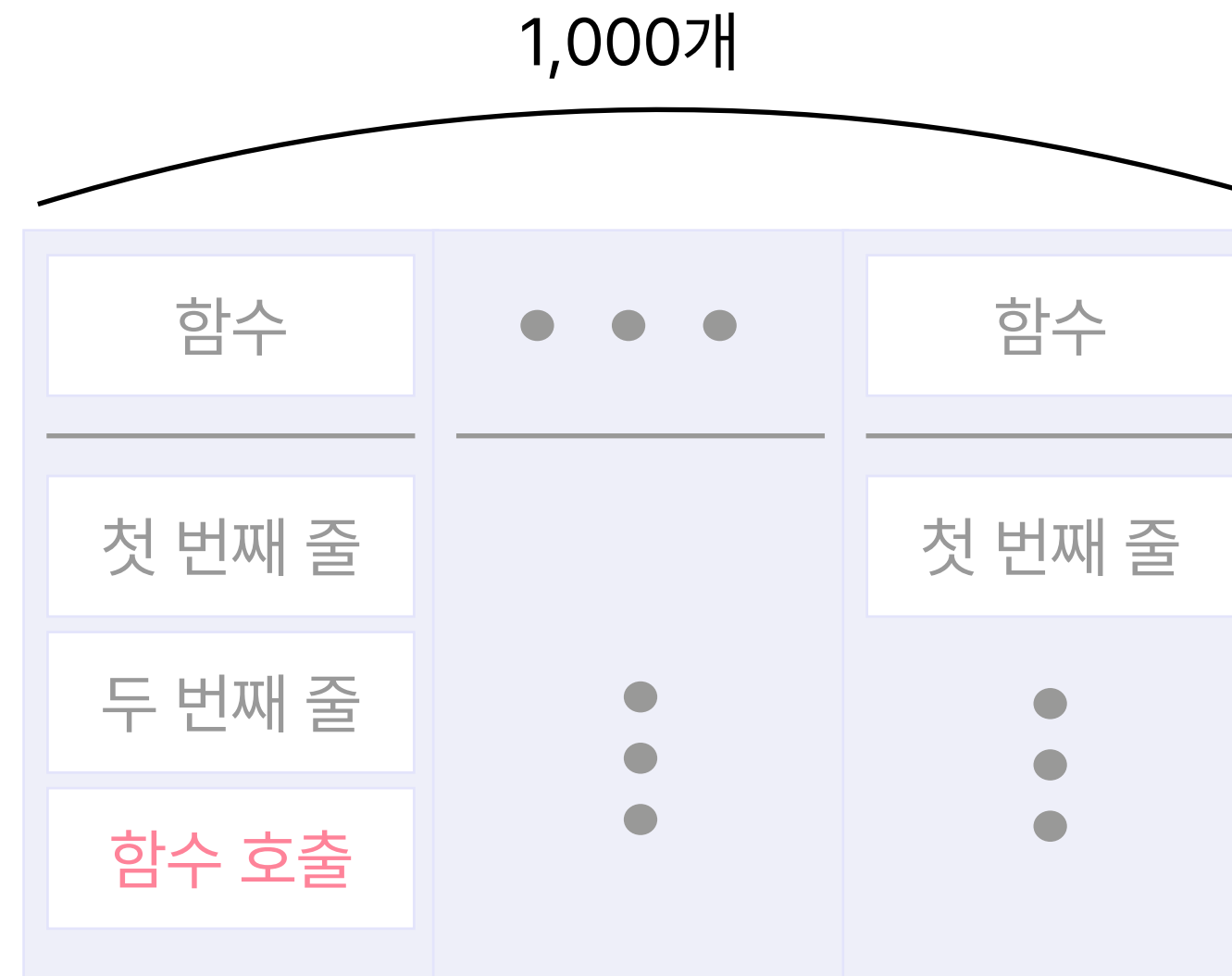
호출 스택에 스택 프레임이 생성되어
해당 함수의 실행 문맥이 저장됨

재귀 호출의 한계 (1) – 시간과 메모리



재귀 호출이 10번 일어나면
스택 프레임이 10개 쌓이고,

재귀 호출의 한계 (1) – 시간과 메모리



재귀 호출이 1,000번 일어나면

1,000개만큼의 스택 프레임이 쌓여 스택 메모리르 더 사용하게 됨



재귀 호출의 한계 (1) – 시간과 메모리

```
def adderForLoop(n):  
    result = 0  
    for i in range(1, n + 1):  
        result += i  
    return result
```

만약 재귀 호출 대신 **for** 반복문을 사용한다면 어떨까?

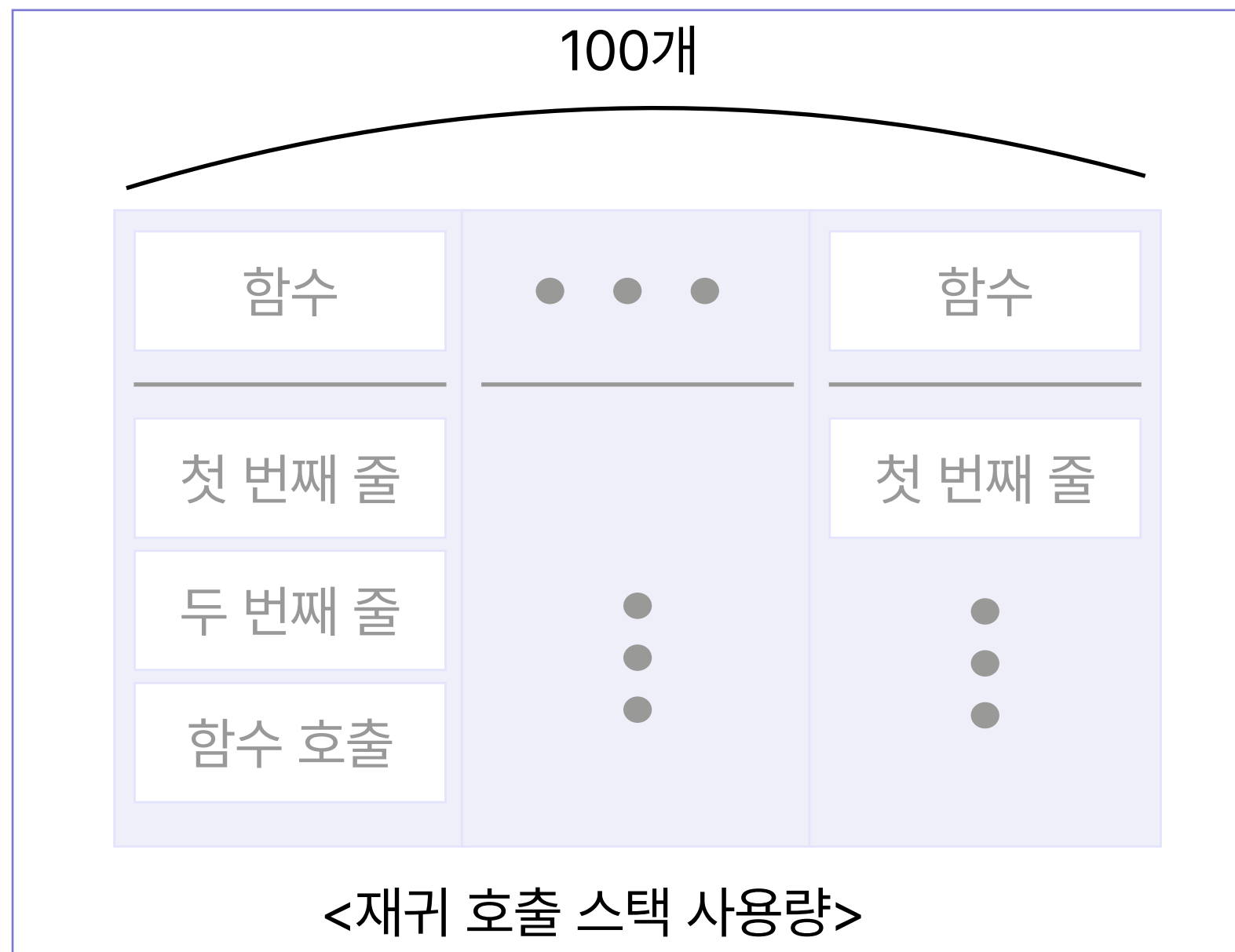


재귀 호출의 한계 (1) – 시간과 메모리

```
def adderForLoop(n):  
    result = 0  
    for i in range(1, n + 1):  
        result += i  
    return result
```

똑같이 n이 100이어도 `adderForLoop()` 함수는
내부적으로 자기 자신을 호출하지 않음

재귀 호출의 한계 (1) – 시간과 메모리



VS



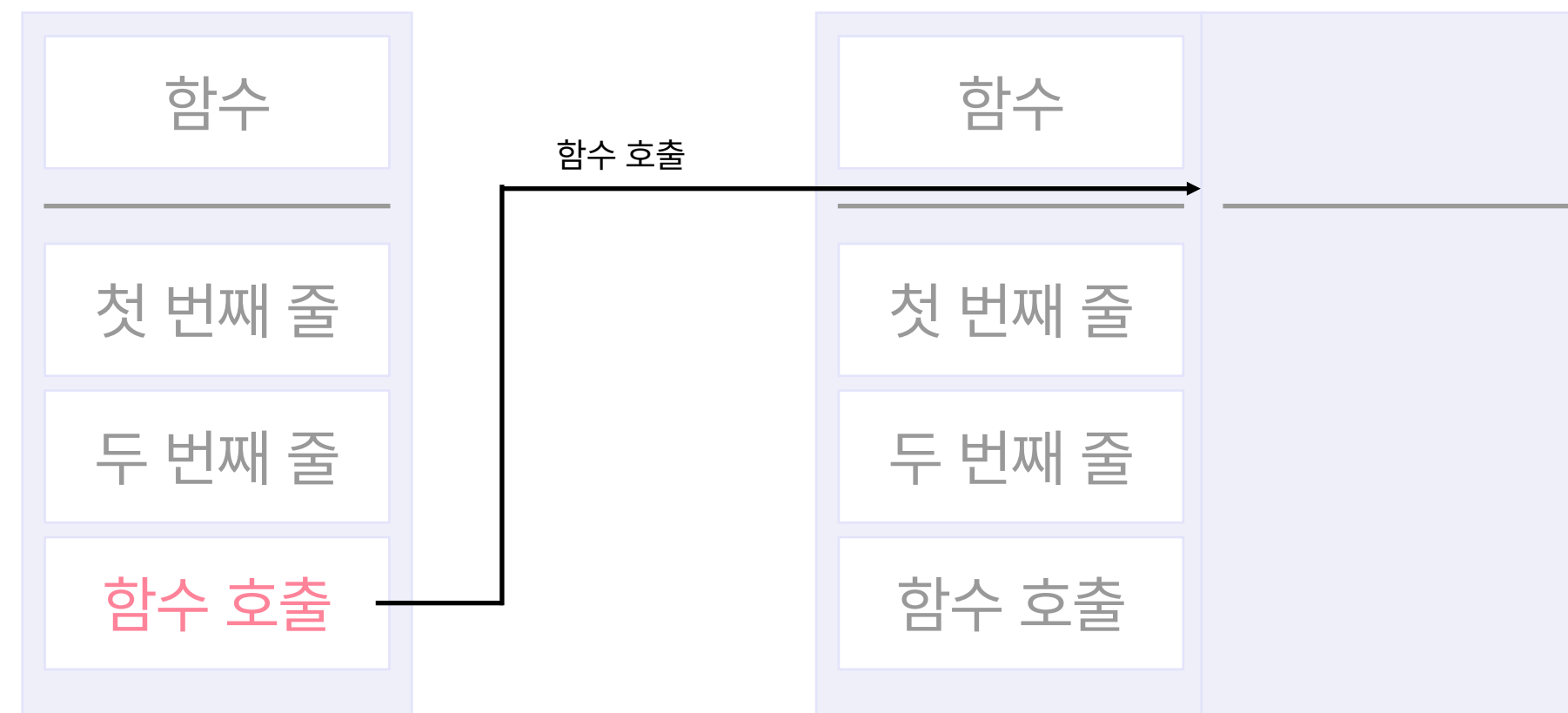


재귀 호출의 한계 (1) – 시간과 메모리

```
def adder(n):  
    if n == 1:  
        return 1  
    else:  
        return n + adder(n - 1)
```

예를 들어, 위 `adder()` 함수가 수행되면
마지막 줄에서 `adder()` 함수를 재귀 호출할 것이며

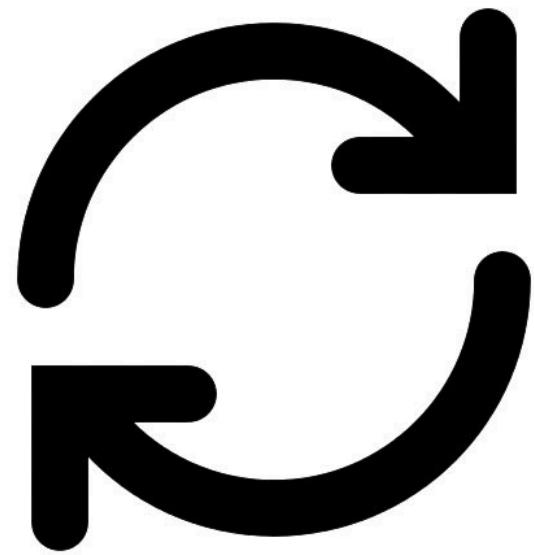
재귀 호출의 한계 (1) – 시간과 메모리



그때마다 새로운 스택 프레임이 쌓이게 됨



재귀 호출의 한계 (1) – 시간과 메모리



반면 **for 반복문**에서는

다음 반복으로 넘어가기 위해 $i += 1$ 만 수행하면 됨



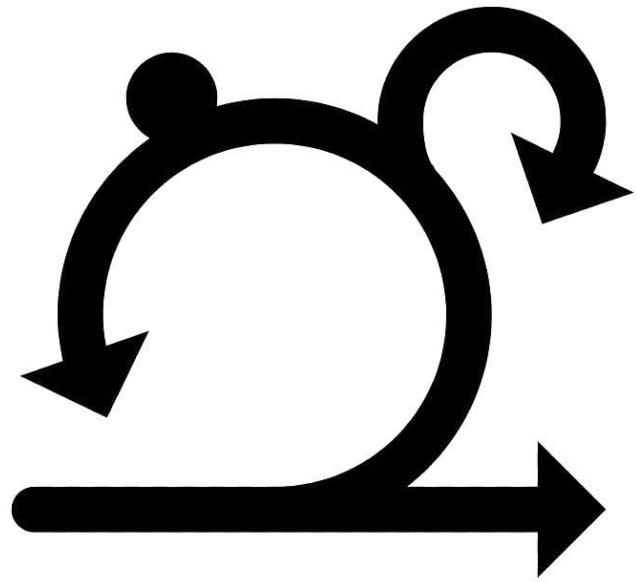
재귀 호출의 한계 (1) – 시간과 메모리

```
def adderForLoop(n):  
    result = 0  
    for i in range(1, n + 1):  
        result += i  
    return result
```

예를 들어, 위 `adderForLoop()` 함수의 반복문을 보면
i 값을 하나씩 늘려가며 다음 반복문을 수행하고 있음

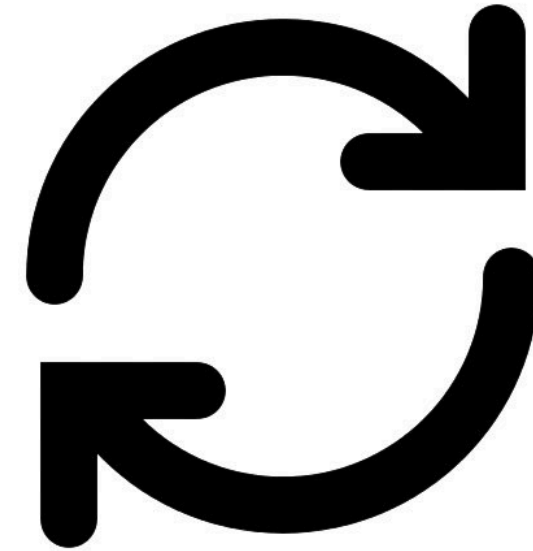


재귀 호출의 한계 (1) – 시간과 메모리



함수 호출 소요 시간

>

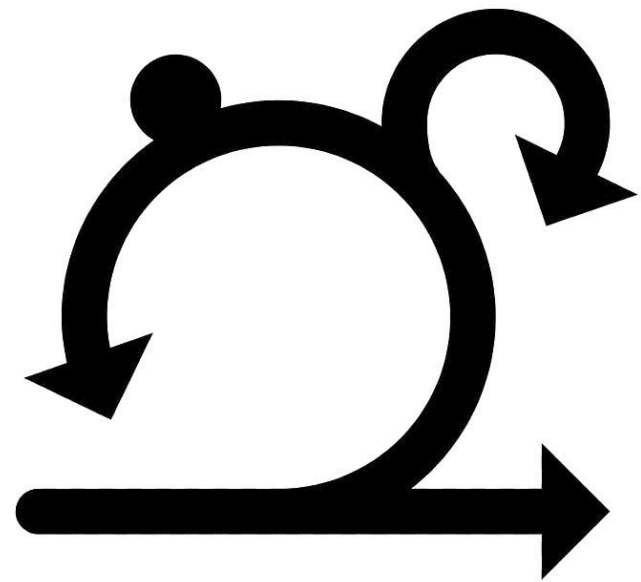


$i += 1$ 소요 시간

`for` 반복문에서 i 에 1을 더하는 데에 드는 시간보다
함수를 여러 번 호출하는 데에 시간이 더 오래 걸리기 때문에

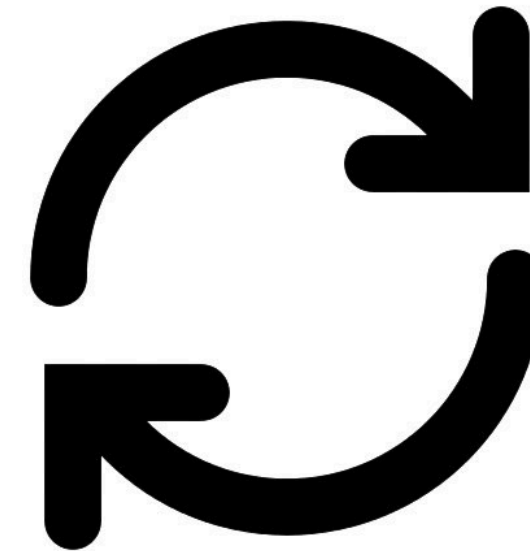


재귀 호출의 한계 (1) – 시간과 메모리



함수 호출 x 100 회

>



$i += 1$ 작업 x 100 회

$i += 1$ 작업을 100번 수행하는 것이
함수를 100번 호출하는 것보다 더 빠름



재귀 호출의 한계 (2) – 반복 계산의 문제

```
def fibonacci(n):    # 바로 앞의 두 항의 합을 구해요.  
    if n == 1 or n == 2:  
        return 1  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)
```

피보나치 함수가 재귀 호출되는 방식을 생각해보면?



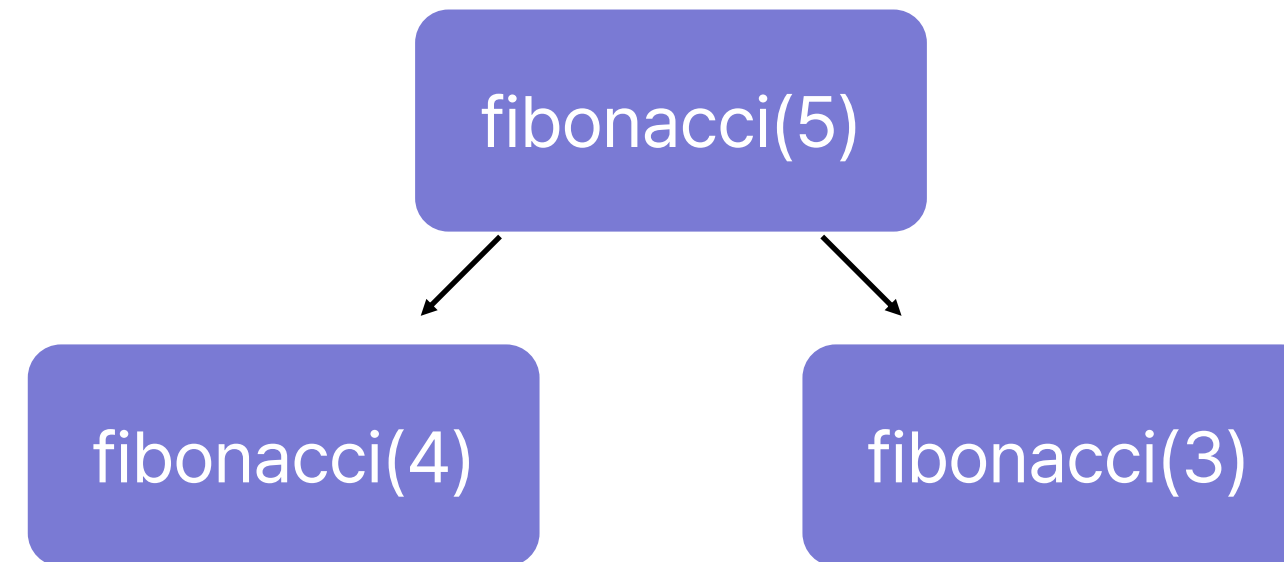
재귀 호출의 한계 (2) – 반복 계산의 문제

fibonacci(5)

매개변수 n 이 5라면

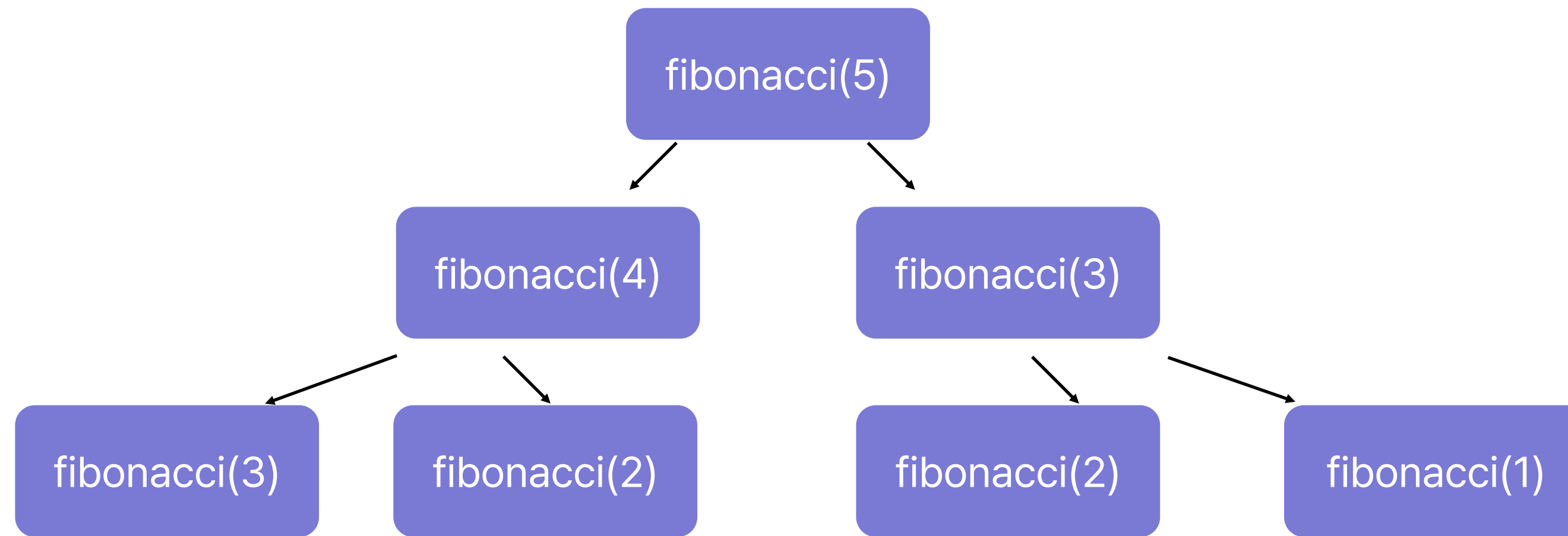


재귀 호출의 한계 (2) – 반복 계산의 문제



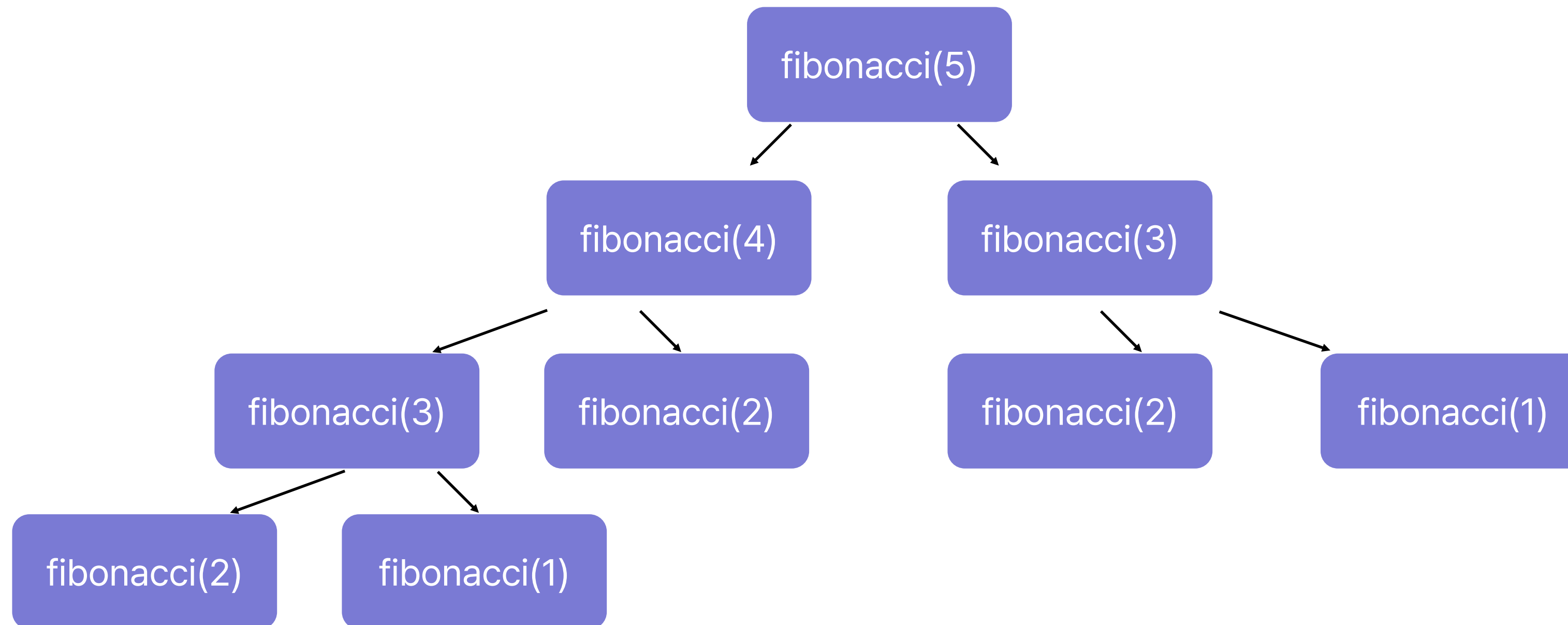
fibonacci(5)를 계산하기 위해
fibonacci(4)와 fibonacci(3)을 계산해야 하며

재귀 호출의 한계 (2) – 반복 계산의 문제



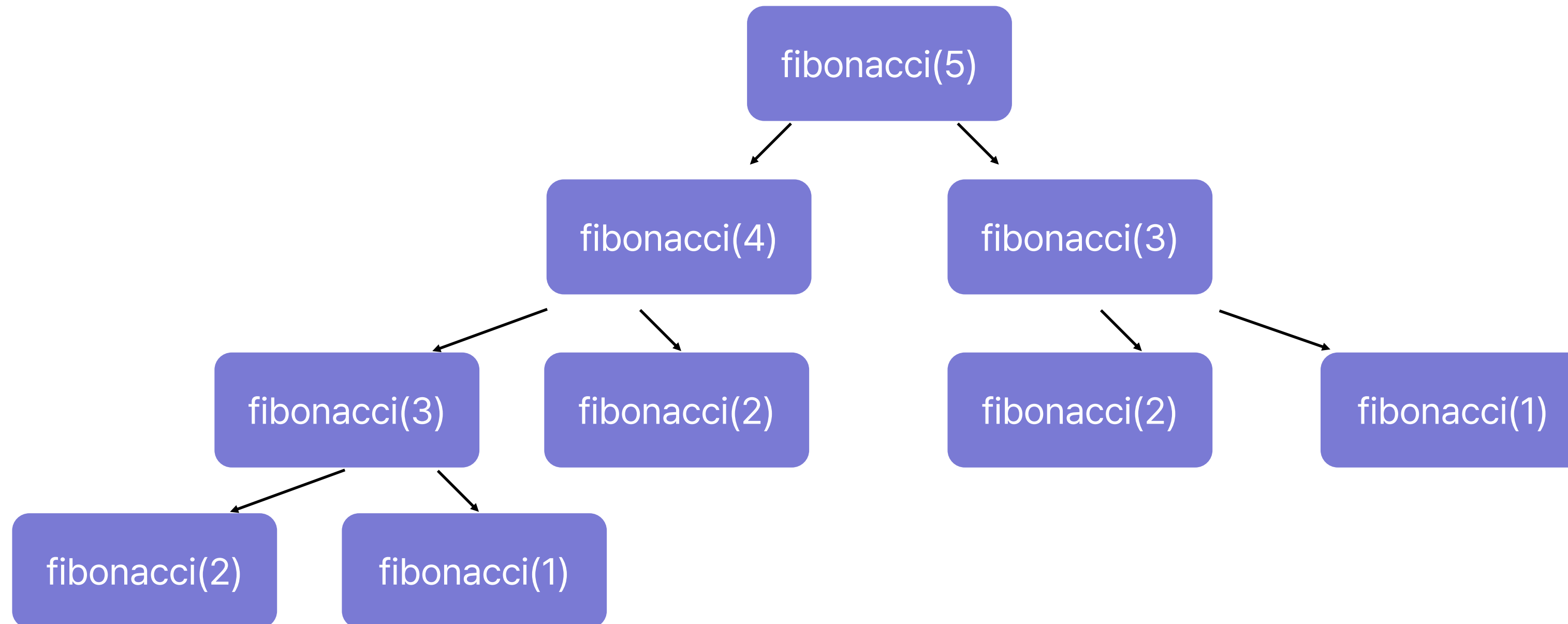
각각은 또 하위 함수의 합으로 표현되어

재귀 호출의 한계 (2) – 반복 계산의 문제



최종적으로는 위의 그림처럼 함수를 호출하게 됨

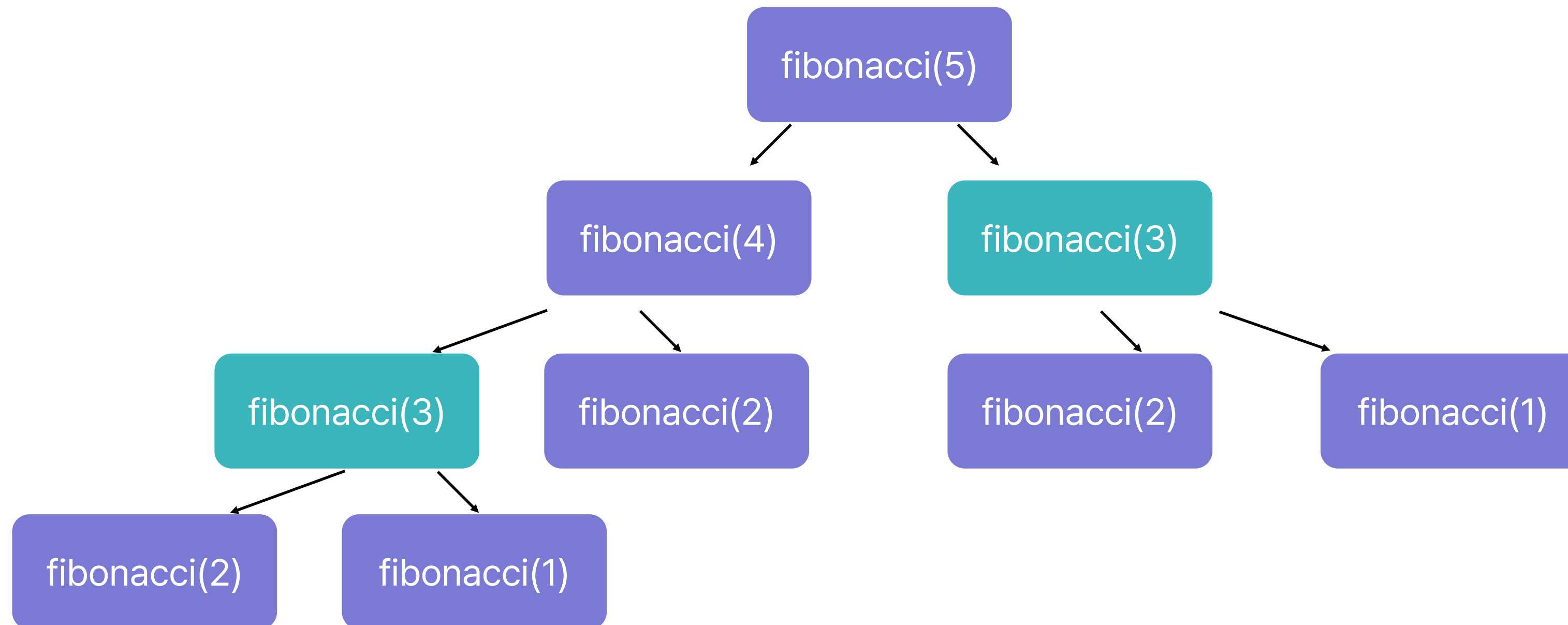
재귀 호출의 한계 (2) – 반복 계산의 문제



잠깐 퀴즈!

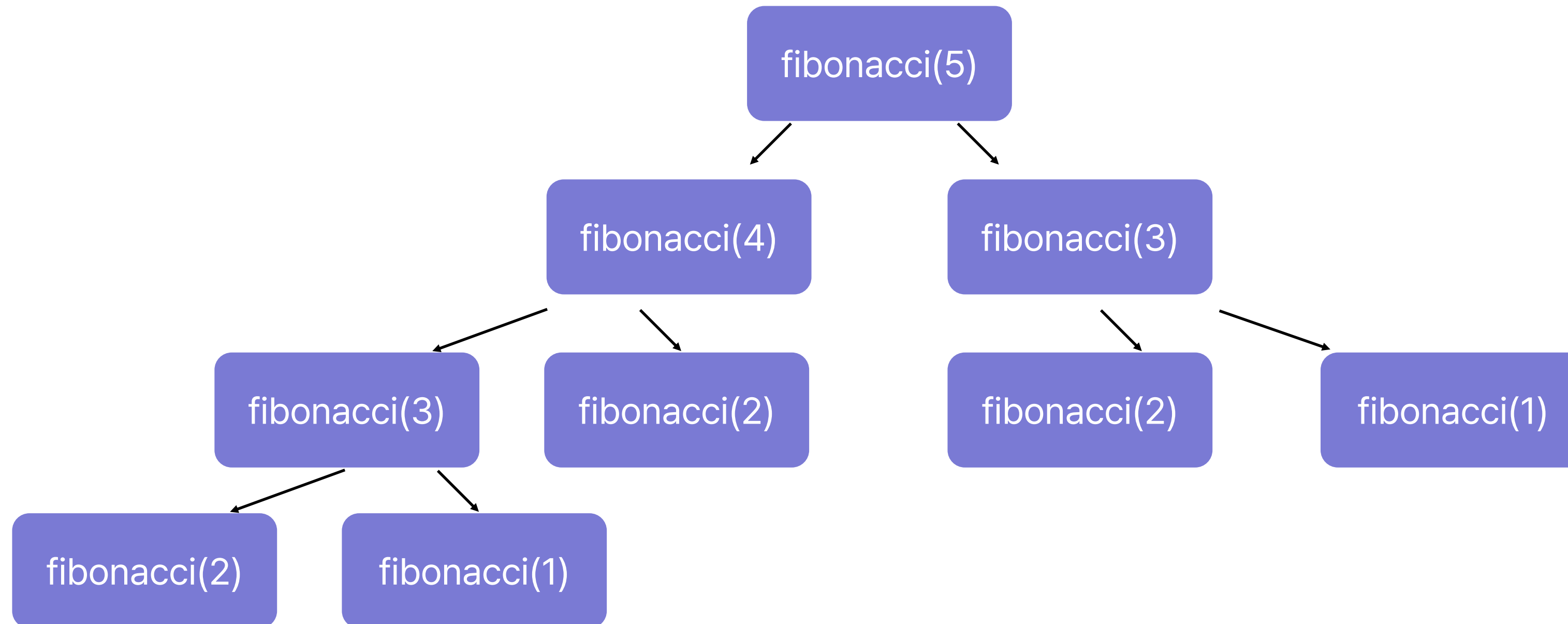
`fibonacci(3)`은 총 몇 번 호출될까?

재귀 호출의 한계 (2) – 반복 계산의 문제



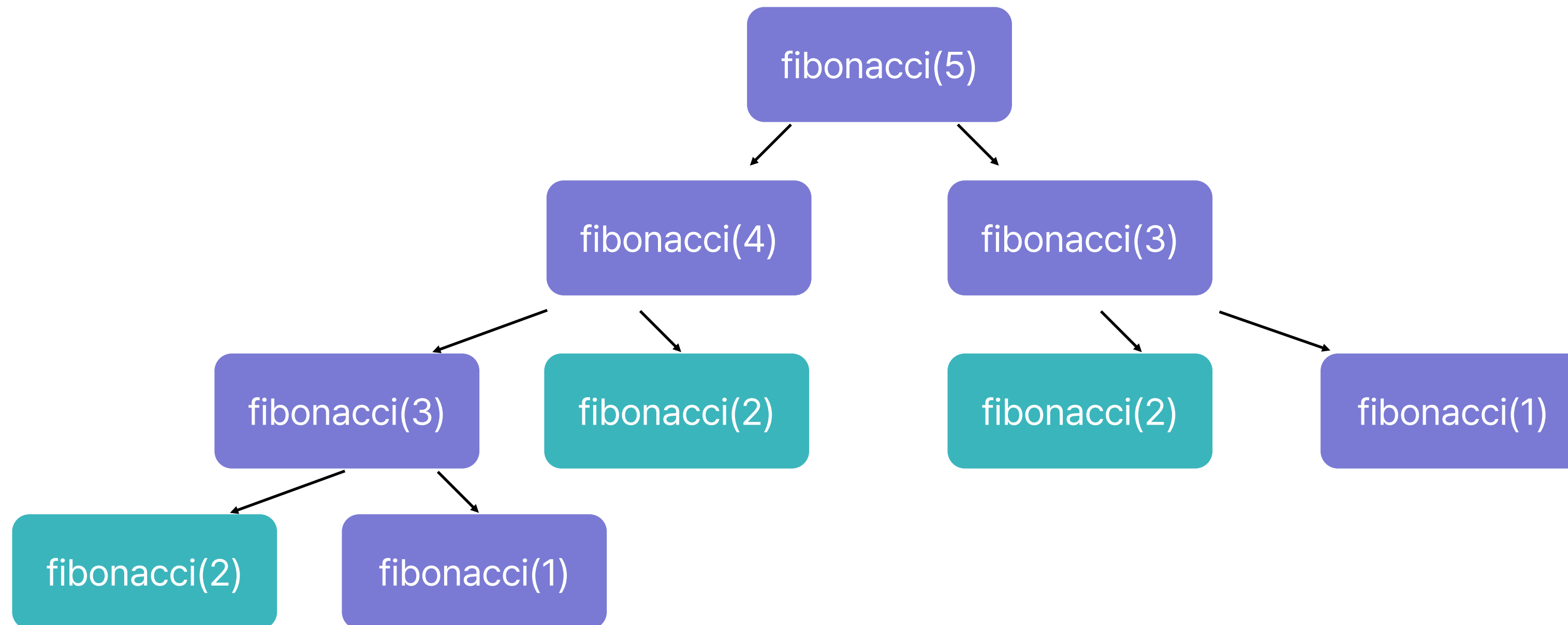
총 2번

재귀 호출의 한계 (2) – 반복 계산의 문제



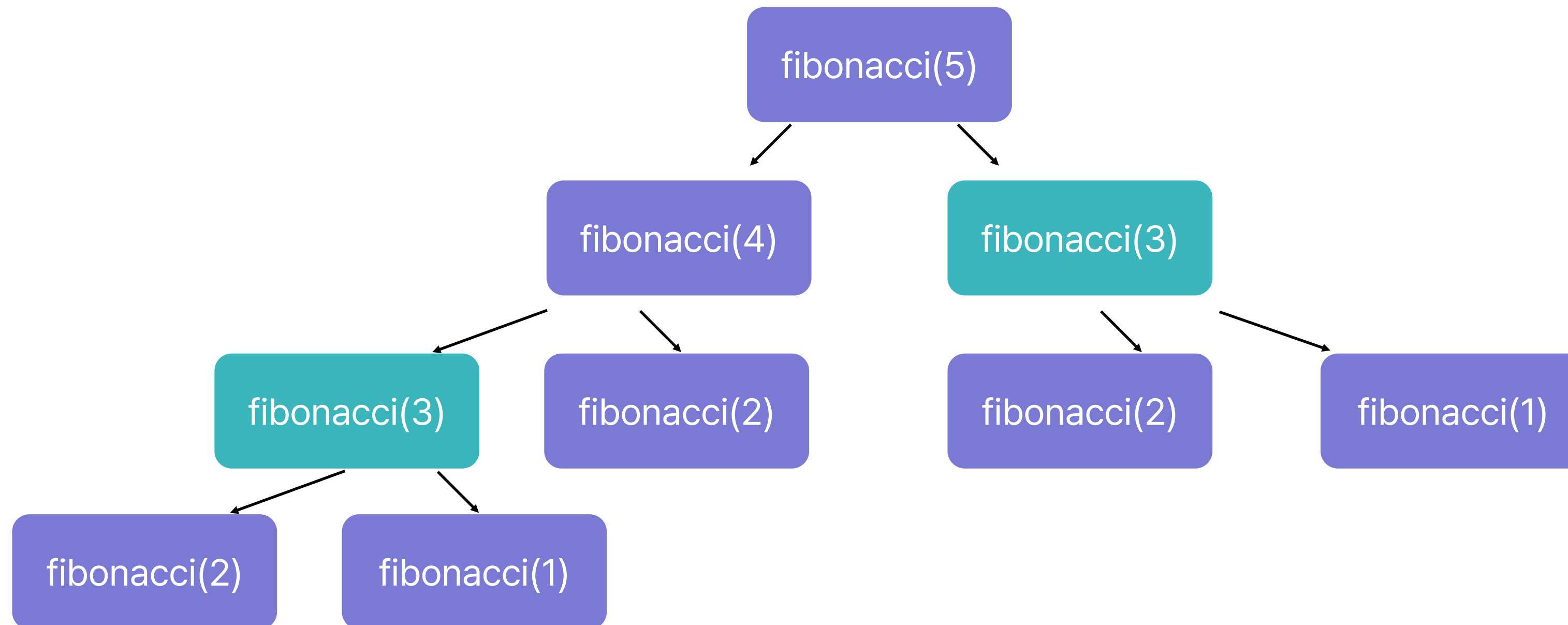
`fibonacci(2)`는 몇 번 호출될까?

재귀 호출의 한계 (2) – 반복 계산의 문제



총 3번

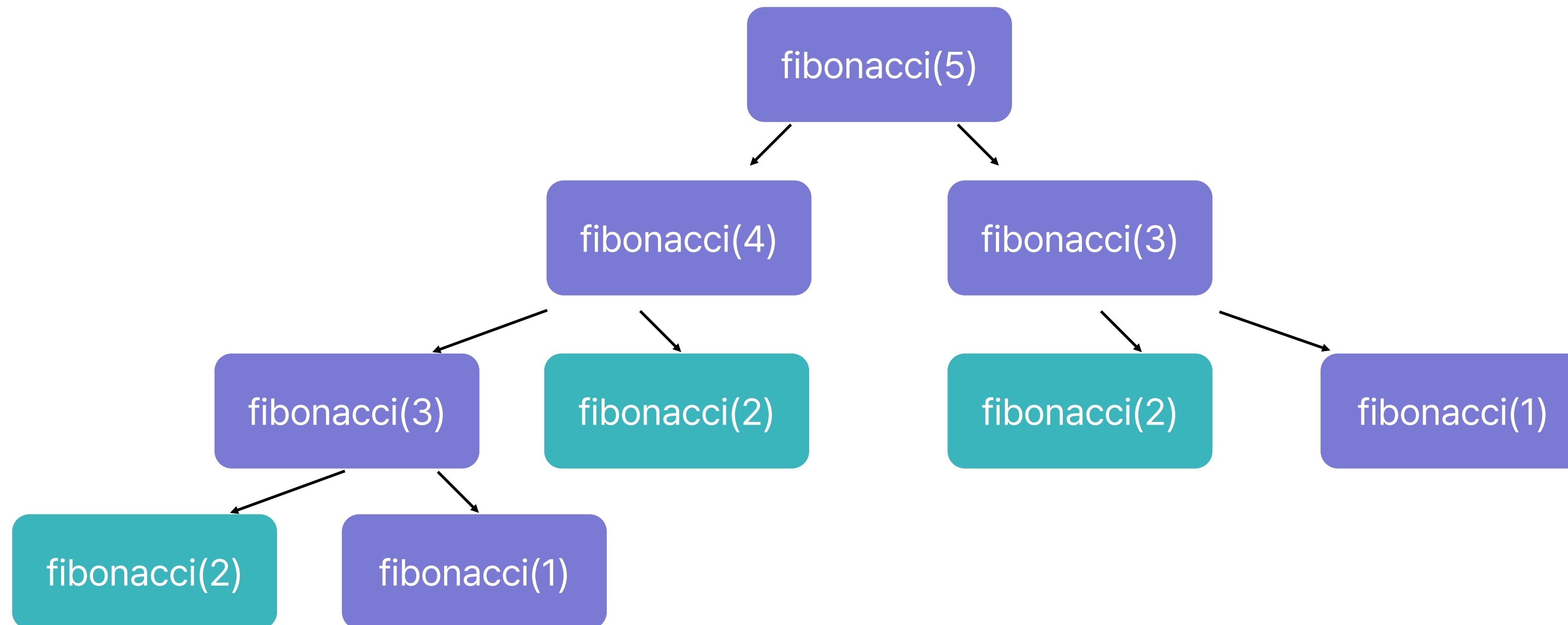
재귀 호출의 한계 (2) – 반복 계산의 문제



fibonacci(3)을 2번 중복해서 계산하지 않고

단 한 번만 계산할 수 있다면 호출에 필요한 시간과 메모리를 아낄 수 있음

재귀 호출의 한계 (2) – 반복 계산의 문제



fibonacci(2)도 3번 중복해서 계산하지 않고
단 한 번만 계산할 수 있다면 시간과 메모리를 아낄 수 있음



재귀 호출의 한계 (2) – 반복 계산의 문제

	n=3	n=5	n=7	n=10	n=20	n=30
fibonacci() 함수 호출 횟수	3	9	25	109	13529	1664079

매개변수 n 이 작을 때는 큰 차이가 없어 보이지만
매개변수 n 이 30만 되어도 함수가 160만 번 이상 호출됨



재귀 호출의 한계 (2) – 반복 계산의 문제

같은 문제를 반복해서 풀지 않으려면
문제에 대한 해답을 기억할 필요가 있음



재귀 호출의 한계 (2) – 반복 계산의 문제

fibonacci(1)

fibonacci(2)

fibonacci(1), fibonacci(2)를 기억한다면



재귀 호출의 한계 (2) – 반복 계산의 문제

fibonacci(1)

fibonacci(2)

fibonacci(3)

fibonacci(3)을 즉시 계산할 수 있고

재귀 호출의 한계 (2) – 반복 계산의 문제

fibonacci(1)

fibonacci(2)

fibonacci(3)

fibonacci(2), fibonacci(3)을 기억한다면



재귀 호출의 한계 (2) – 반복 계산의 문제

fibonacci(1)

fibonacci(2)

fibonacci(3)

fibonacci(4)

fibonacci(4)를 즉시 계산할 수 있으며

재귀 호출의 한계 (2) – 반복 계산의 문제

fibonacci(1)

fibonacci(2)

fibonacci(3)

fibonacci(4)

fibonacci(3), fibonacci(4)를 기억한다면



재귀 호출의 한계 (2) – 반복 계산의 문제

fibonacci(1)

fibonacci(2)

fibonacci(3)

fibonacci(4)

fibonacci(5)

fibonacci(5)를 즉시 계산할 수 있음

재귀 호출의 한계 (2) – 반복 계산의 문제

fibonacci(1)

fibonacci(2)

fibonacci(3)

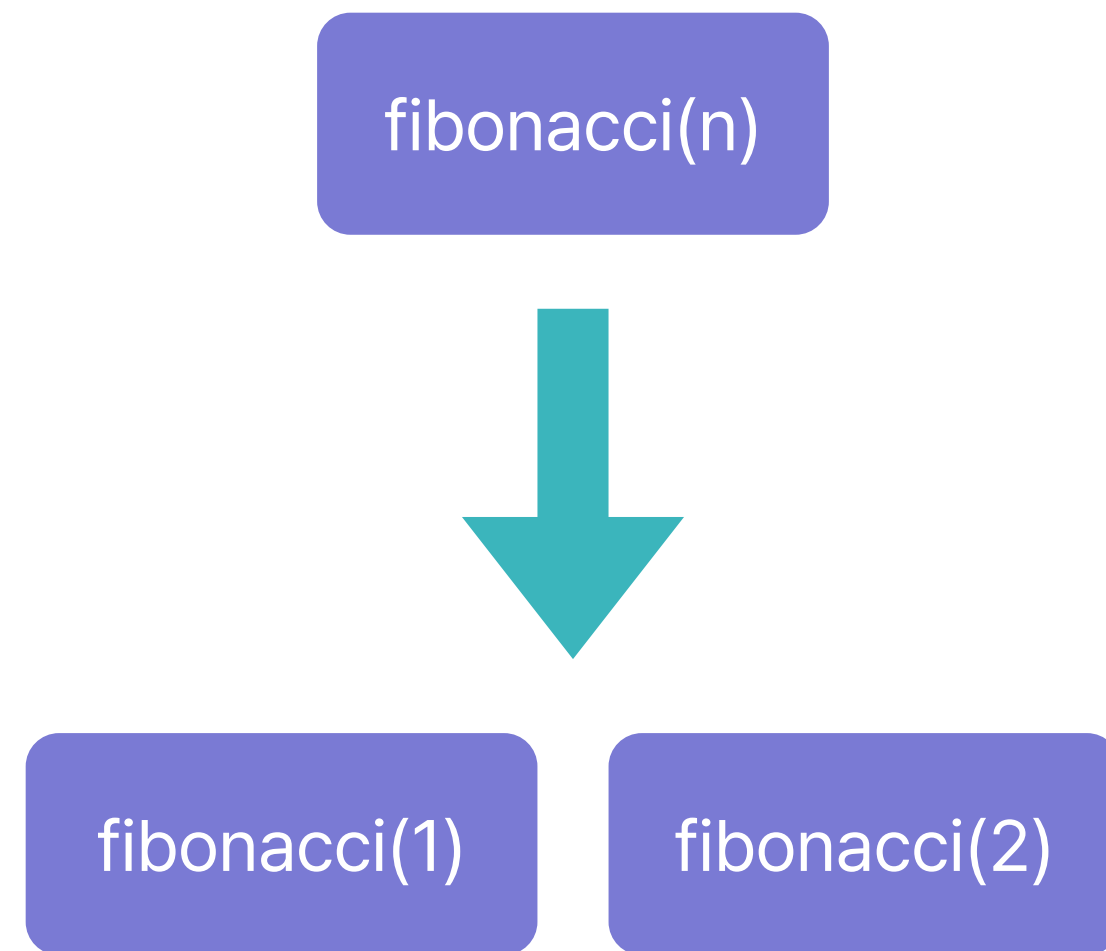
fibonacci(4)

fibonacci(5)

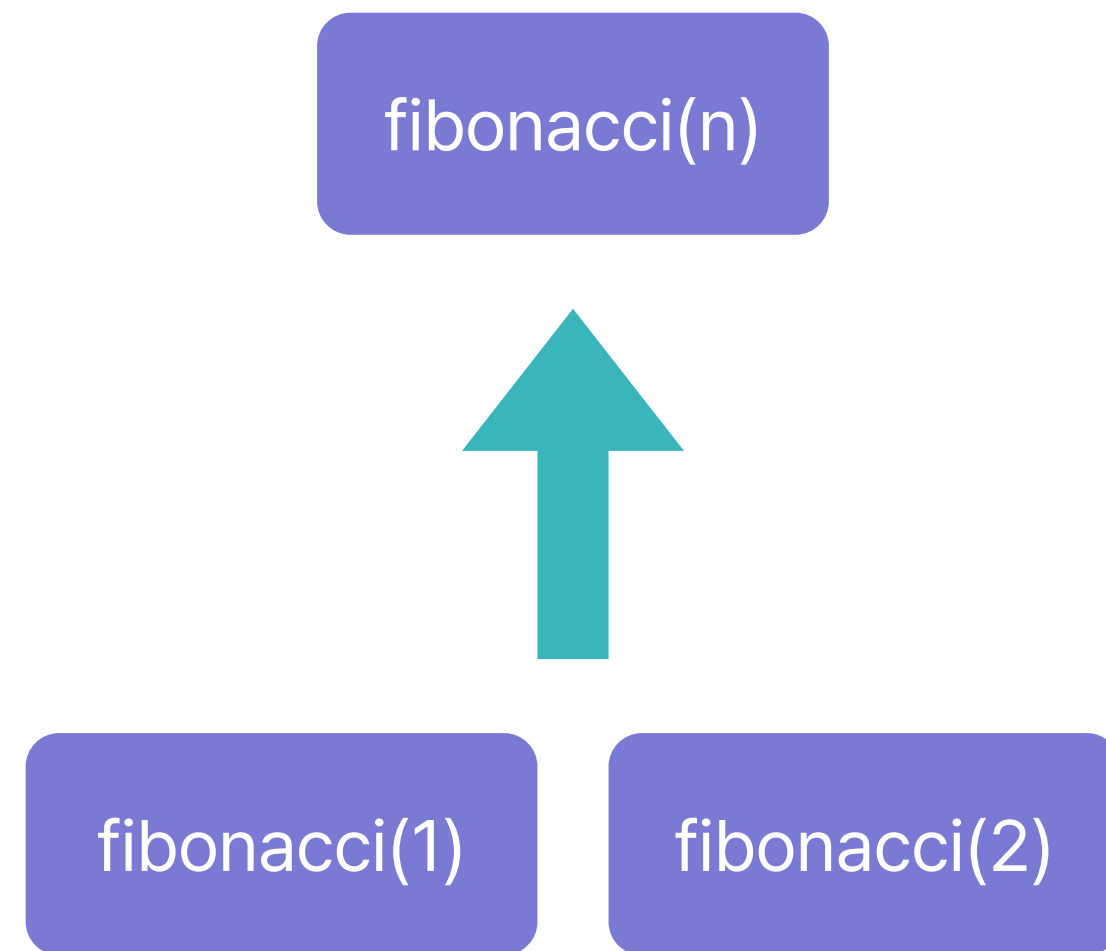


이렇게 되면, 함수 호출에 필요한 메모리도 아낄 수 있고
똑같은 문제는 단 한 번만 풀게 되니 알고리즘은 더 빨라짐

재귀 호출의 한계 (2) – 반복 계산의 문제

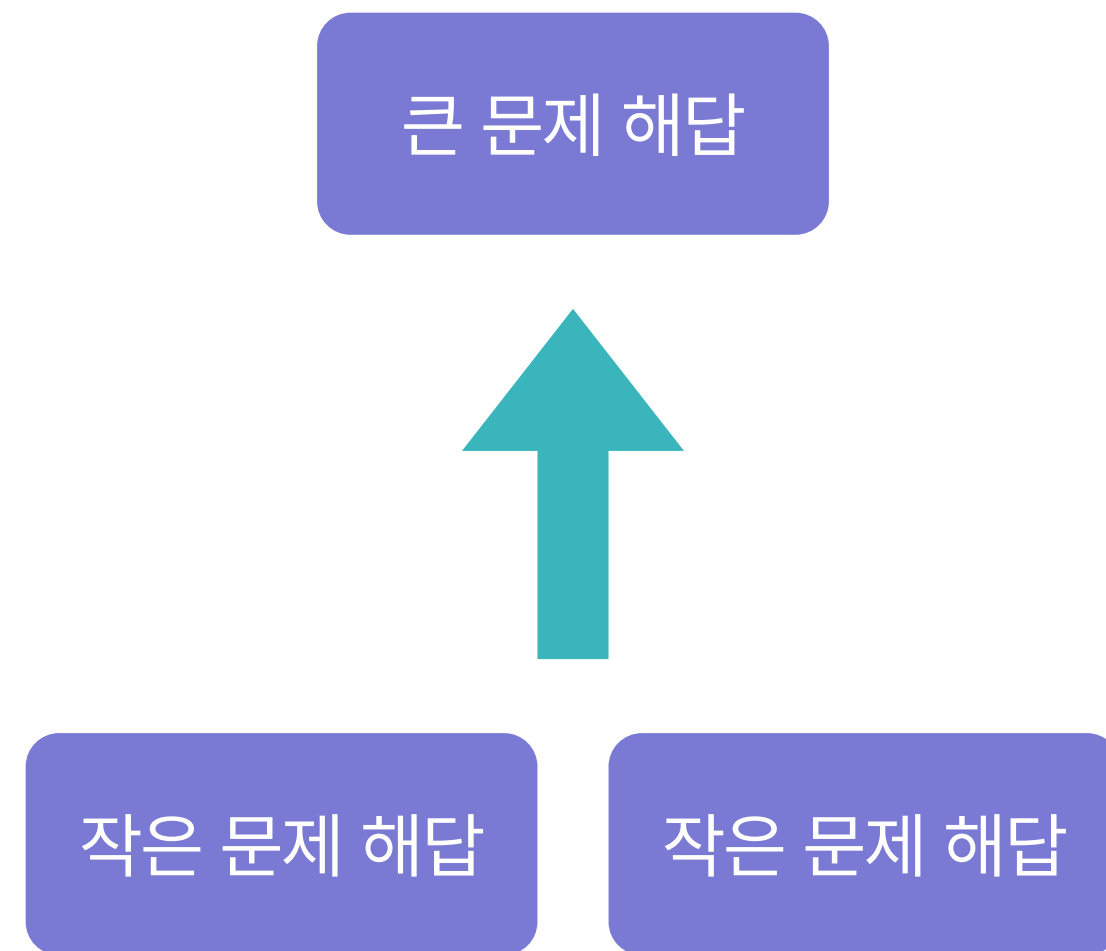


재귀 호출은 n에서 1로 내려가는 **하향식 프로그래밍 기법**이었다면



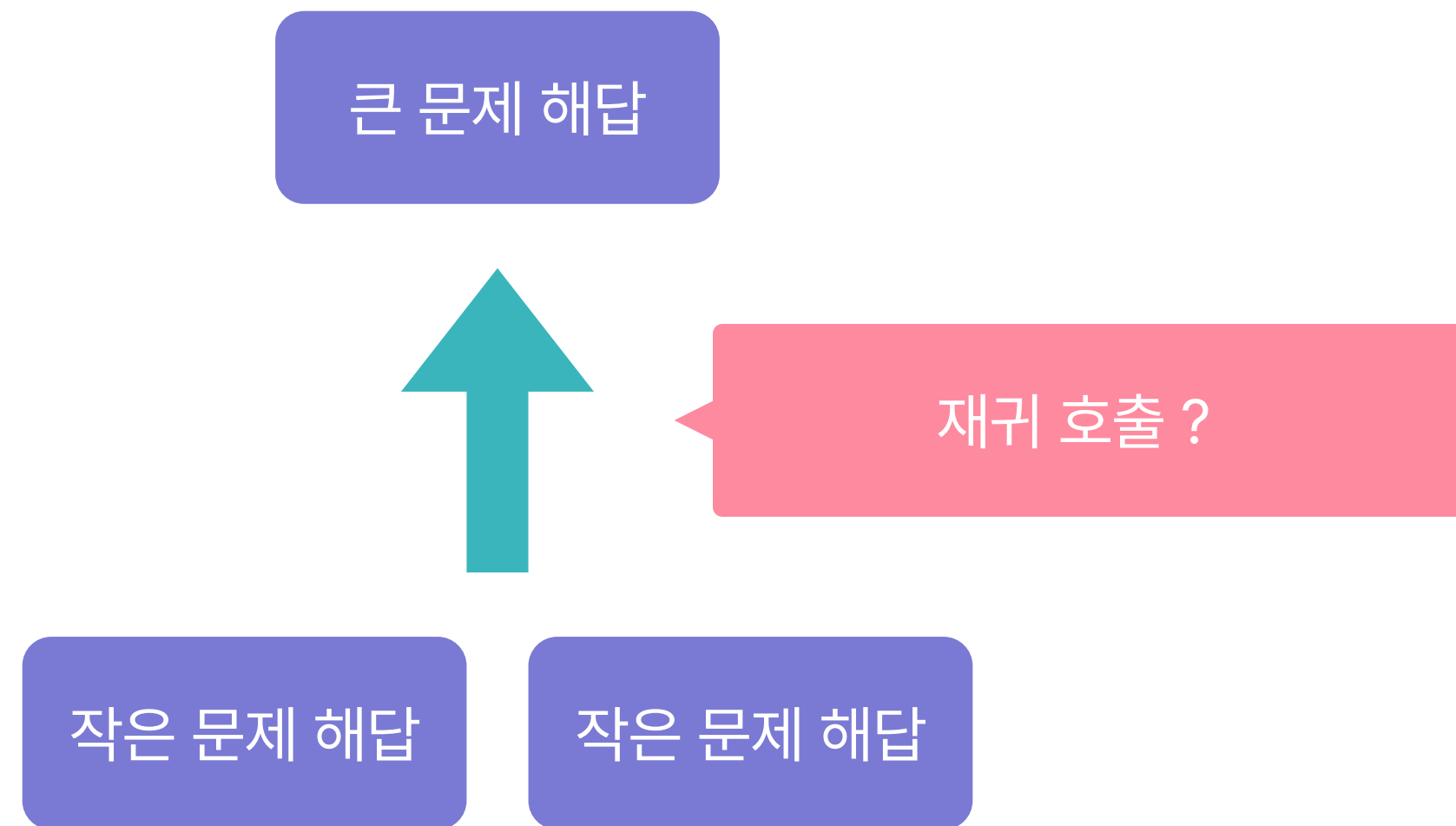
우리가 개선하려는 방식은 작은 문제의 해답을 기억하면서
큰 문제의 답을 구해나간다는 점에서 **상향식 프로그래밍 기법**

Dynamic Programming



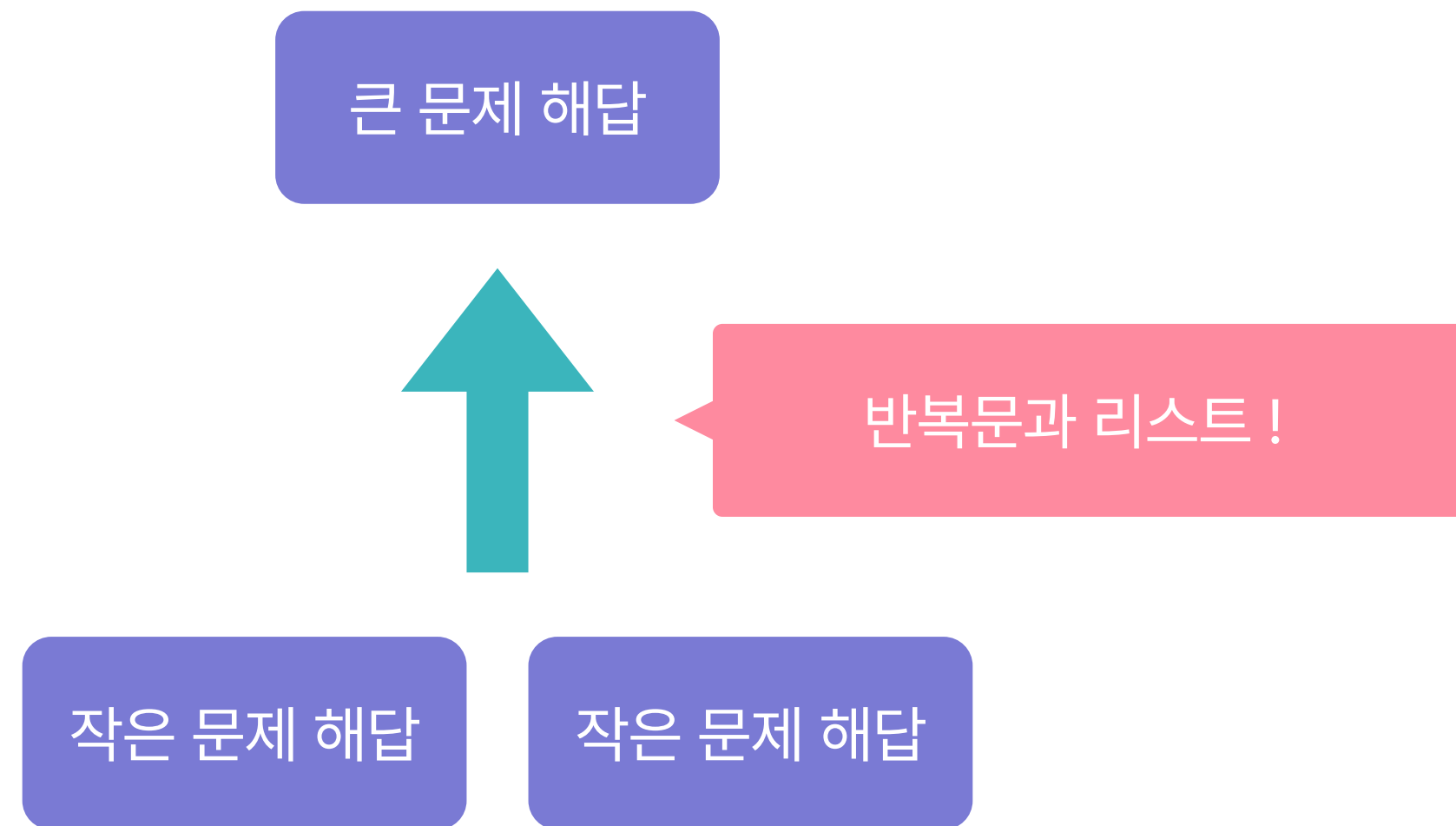
작은 문제의 해답을 기억하며 큰 문제를 향해 상향식으로 나아가는 방식,
이것이 바로 동적 계획법의 핵심

동적 계획법 (Dynamic Programming)



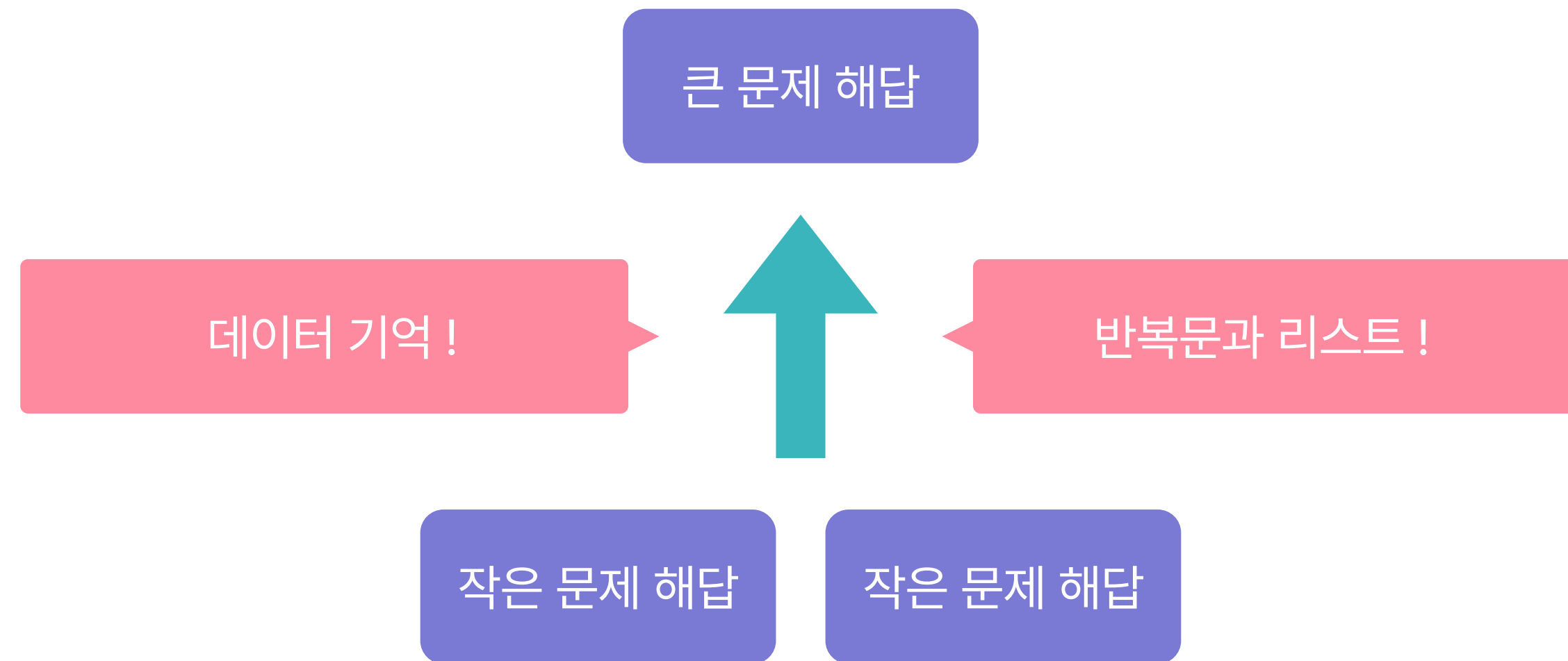
시간과 메모리가 많이 필요한 재귀 호출 함수 대신

동적 계획법 (Dynamic Programming)



하나의 함수 내에서
반복문과 리스트를 활용해서 문제를 해결하는 것

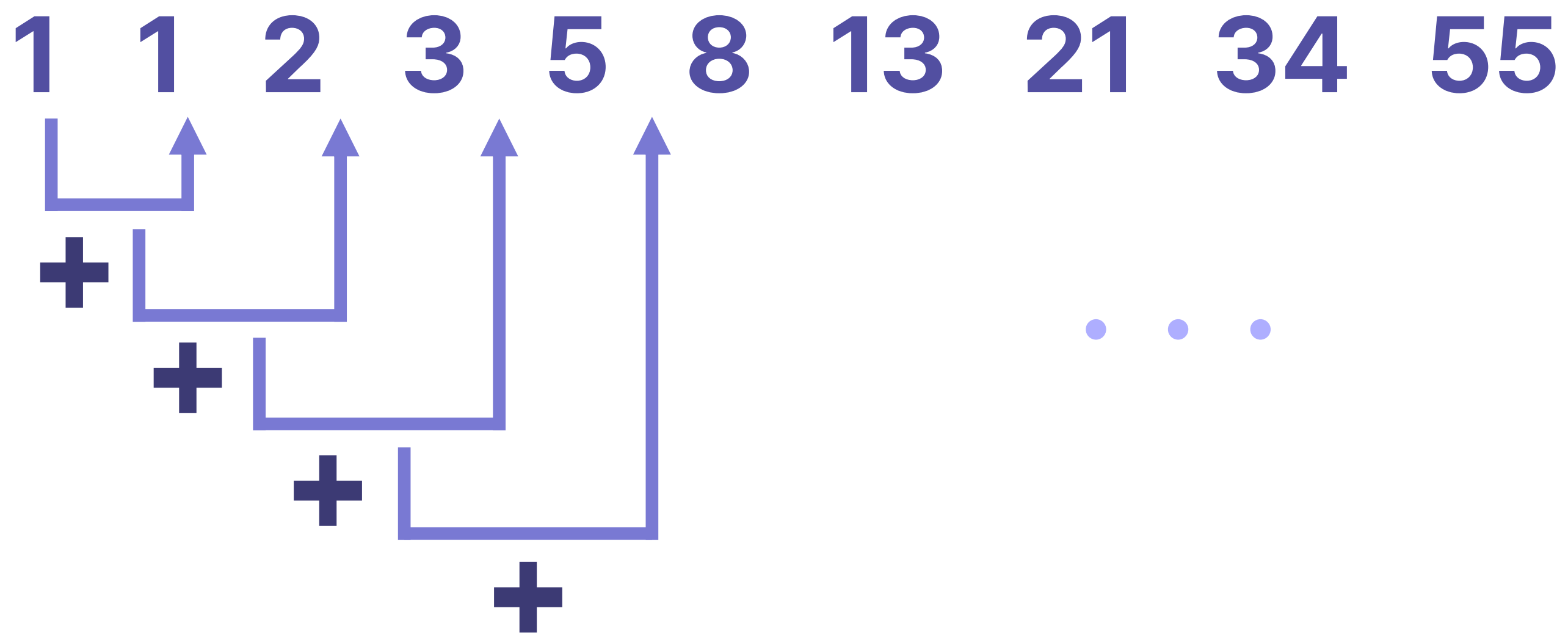
동적 계획법 (Dynamic Programming)



리스트에 작은 문제의 해답을 메모해두고
이 기억을 활용해 큰 문제를 풀어나가는 방식



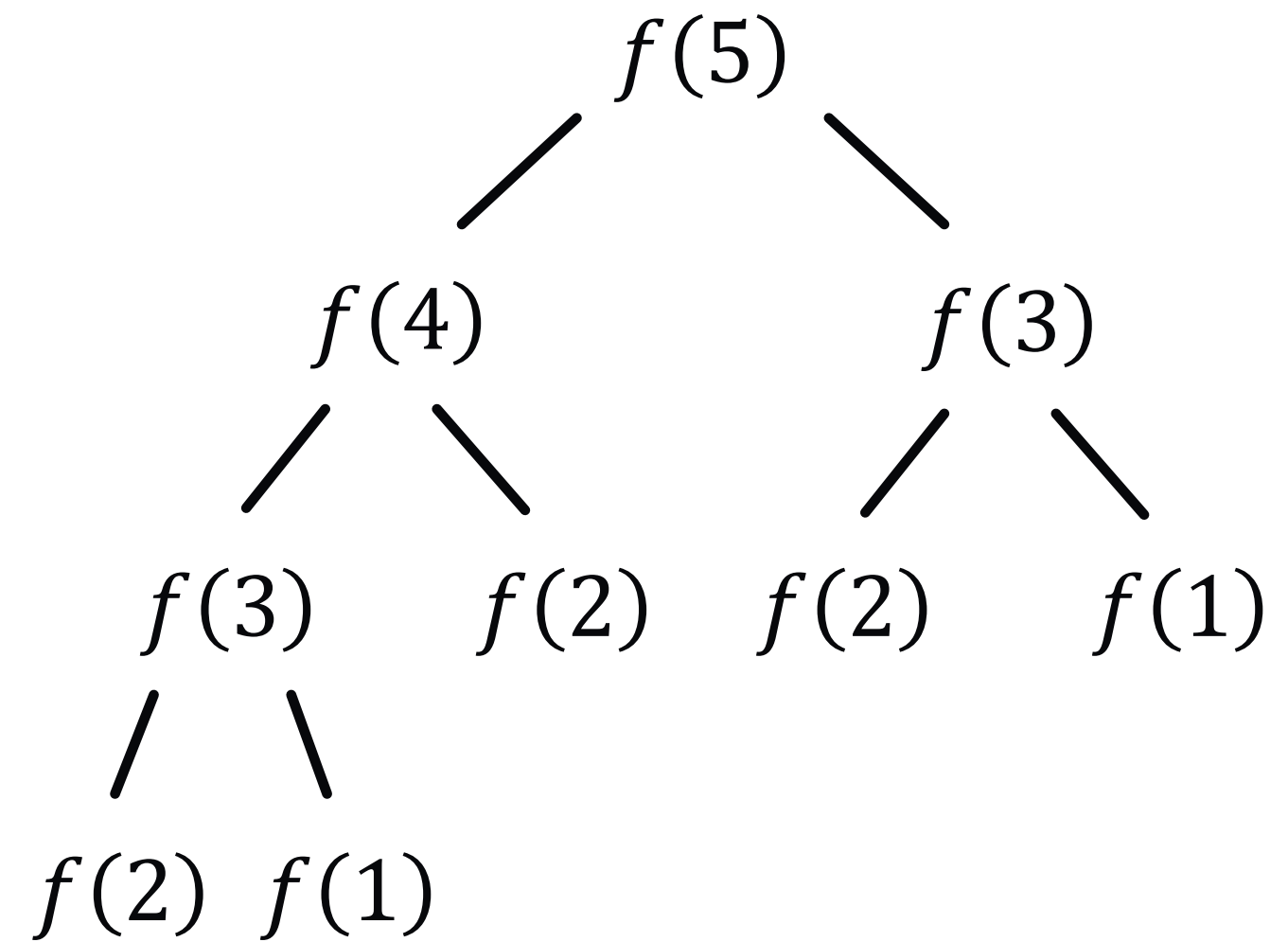
피보나치 수열





재귀를 활용한 피보나치 수열

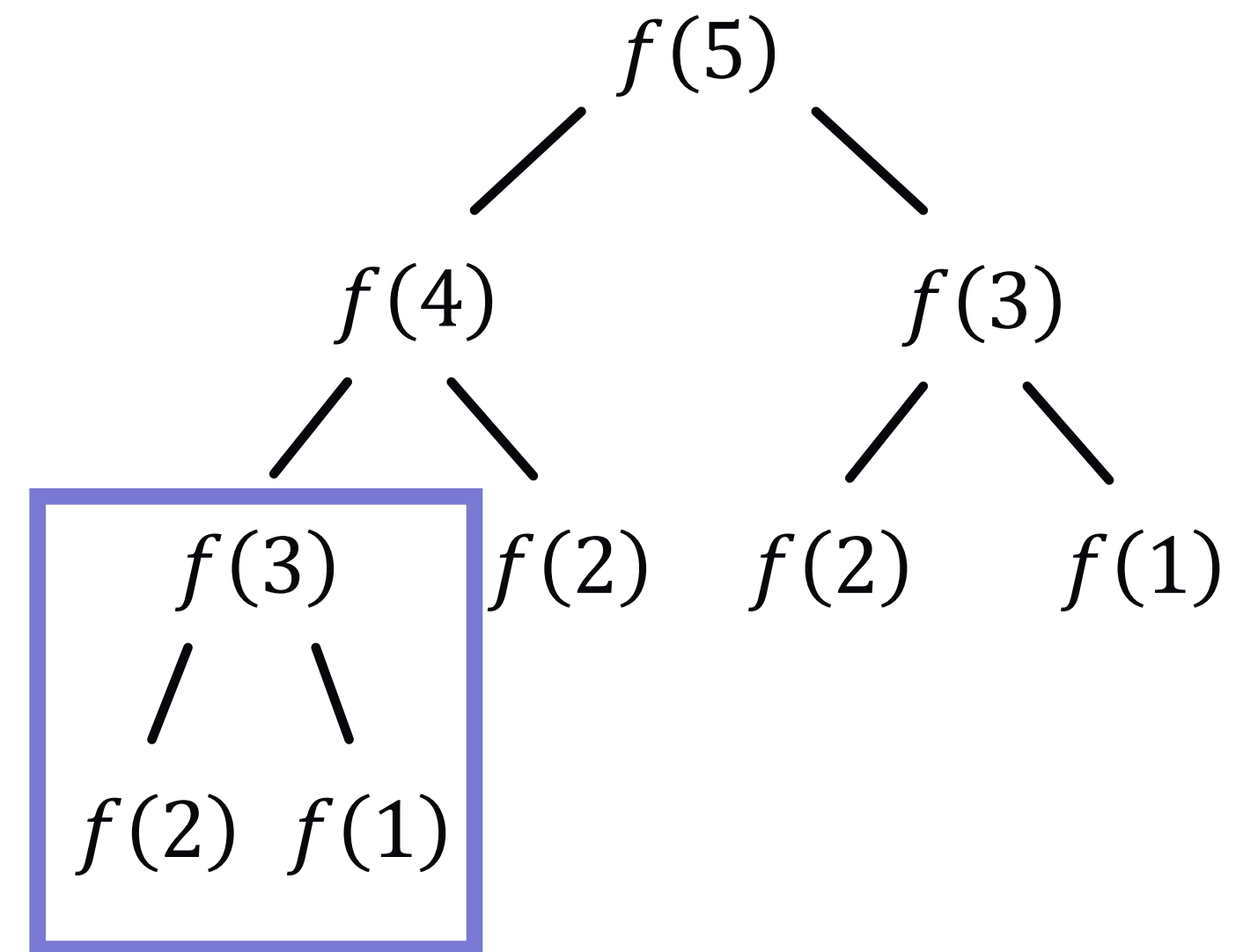
```
def fibo(n):  
    if n < 3:  
        return 1  
    else:  
        return fibo(n-1) + fibo(n-2)
```





재귀를 활용한 피보나치 수열

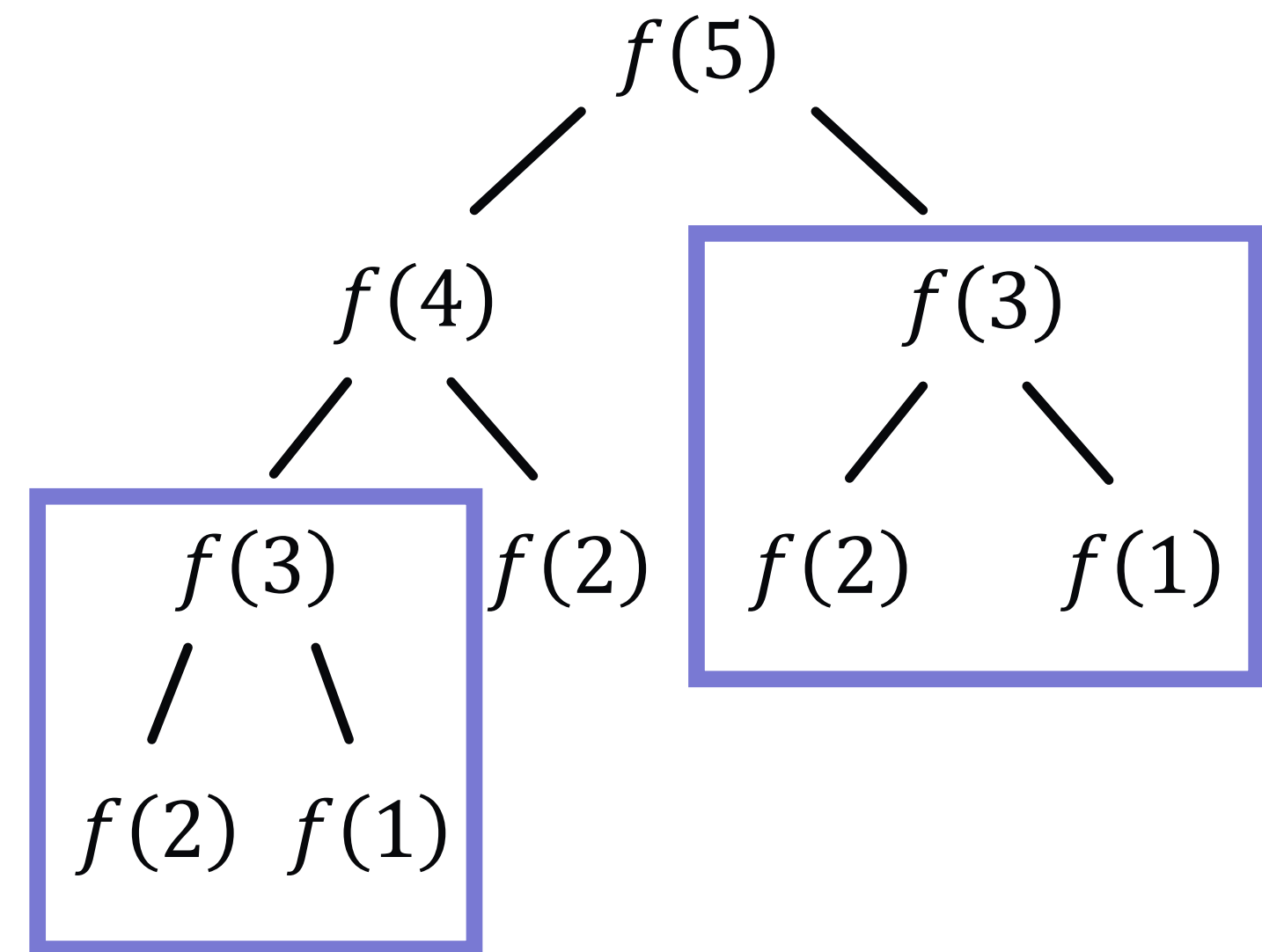
```
def fibo(n):  
    if n < 3:  
        return 1  
    else:  
        return fibo(n-1) + fibo(n-2)
```





재귀를 활용한 피보나치 수열

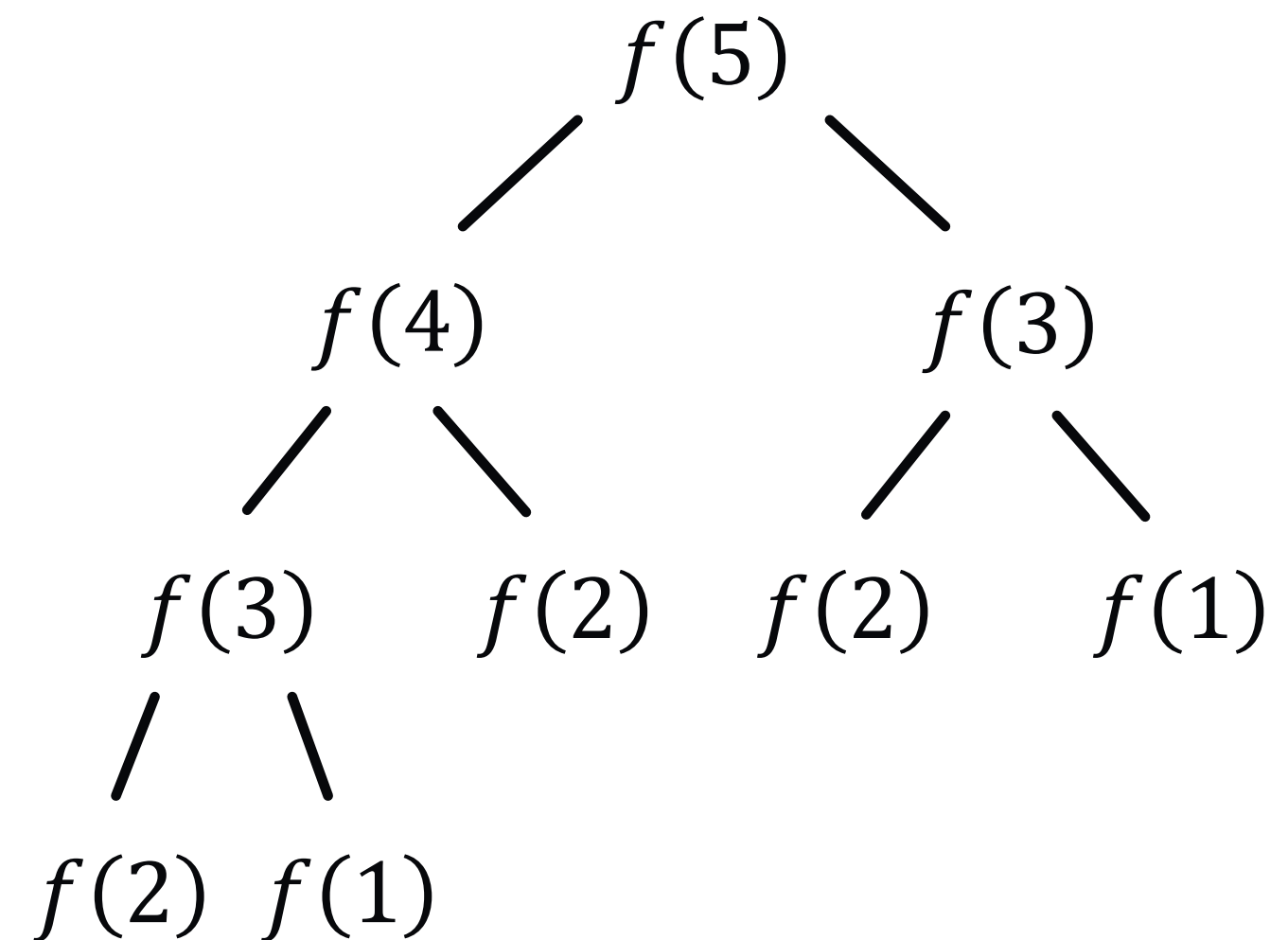
```
def fibo(n):  
    if n < 3:  
        return 1  
    else:  
        return fibo(n-1) + fibo(n-2)
```



동적계획법 (Dynamic Programming)

복잡한 문제를 간단한 **여러 개의 하위 문제로 나누어** 푸는 방법
이 때, 하위 문제의 답을 저장하여 **중복 연산을 하지 않음**

```
def fibo(n):  
    if n < 3:  
        return 1  
    else:  
        return fibo(n-1) + fibo(n-2)
```



동적계획법 (Dynamic Programming)

복잡한 문제를 간단한 **여러 개의 하위 문제로 나누어** 푸는 방법
이 때, 하위 문제의 답을 저장하여 **중복 연산을 하지 않음**

```
fibonacci = {1: 1, 2: 1}
```

cache

```
def fibo(n):
```

```
    if n in fibonacci:
```

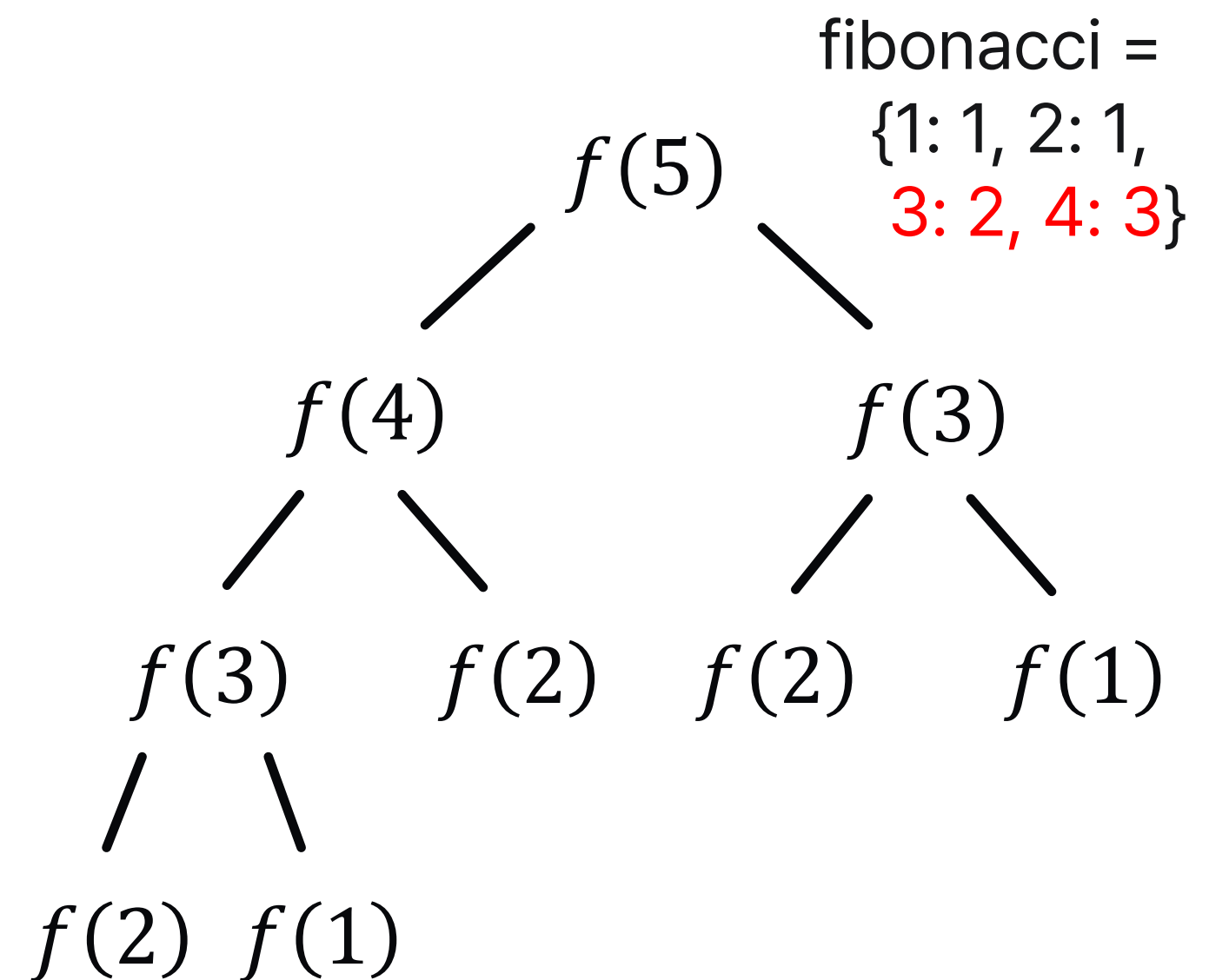
```
        return fibonacci[n]
```

memoization

```
    else:
```

```
        fibonacci[n] = fibo(n-1) + fibo(n-2)
```

```
    return fibonacci[n]
```





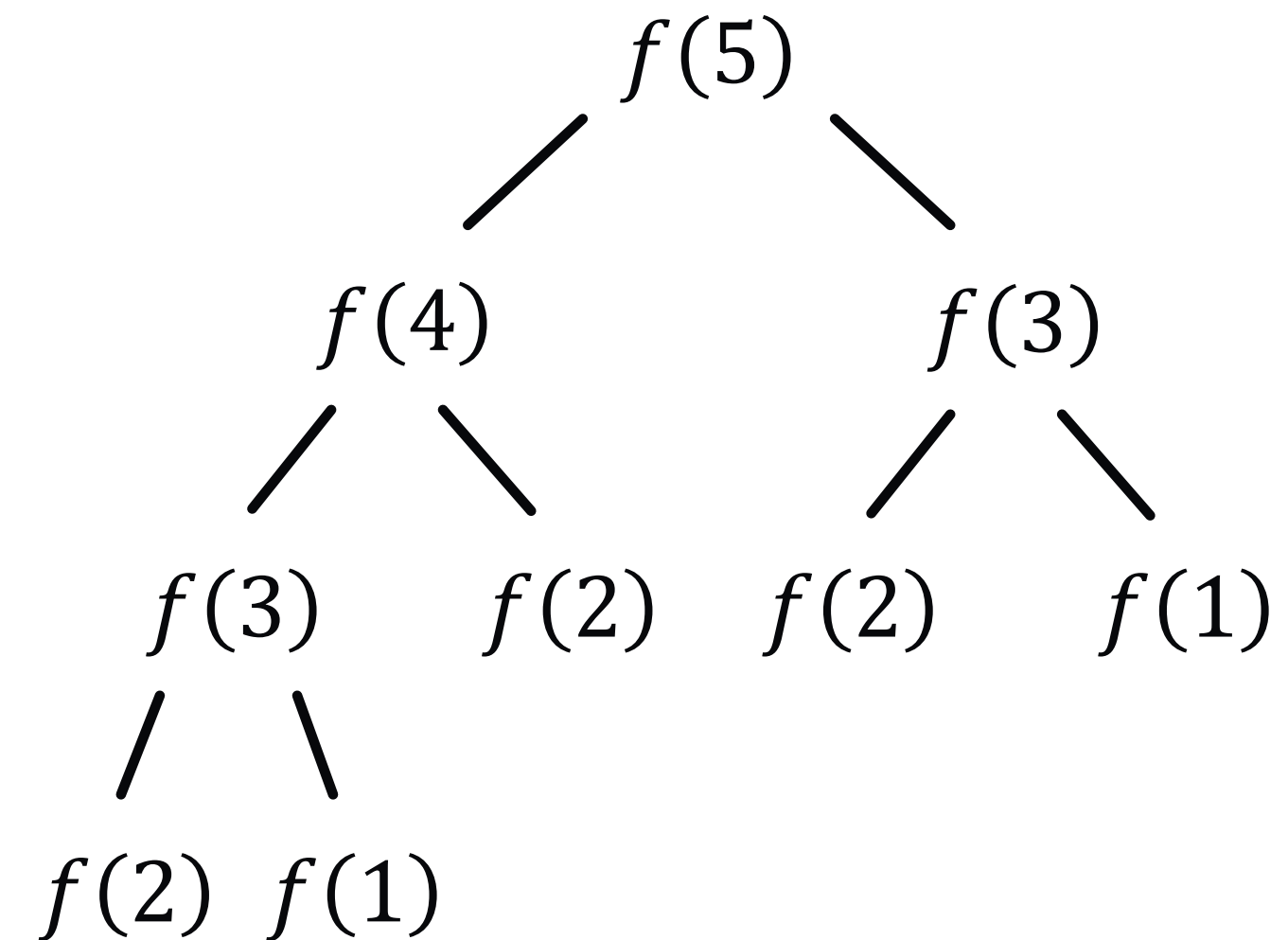
동적계획법 문제의 특성

중복되는 부분문제 (overlapping subproblems)

작은 하위문제들이 중복되어 나타난다.

최적 부분 구조 (optimal substructure)

최적해는 부분 문제의 최적해로부터 구할 수 있다.





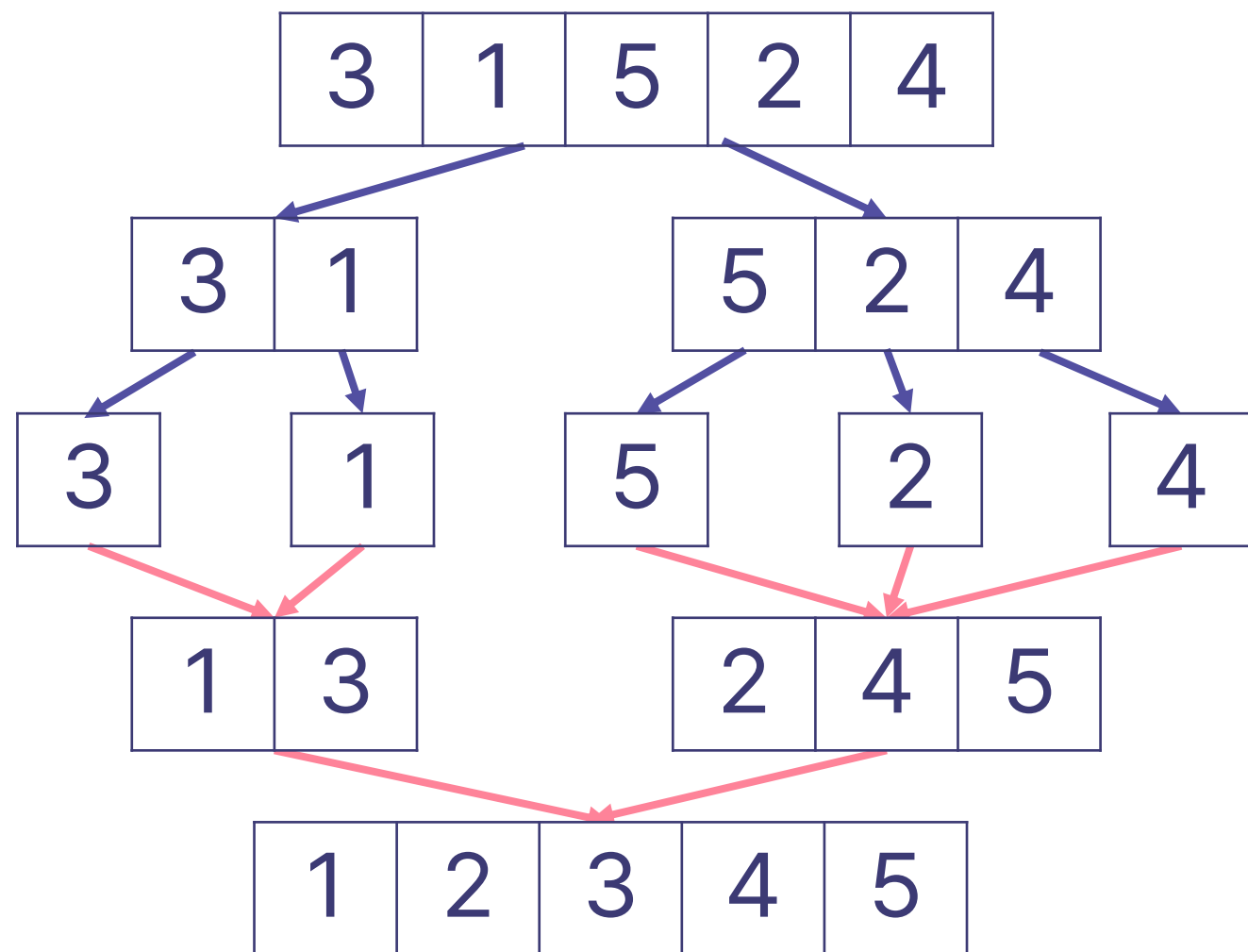
분할정복법과 동적계획법의 차이

분할정복법은 중간 결과를 저장하여 효율성을 높인다는 점에서 차이가 있다!

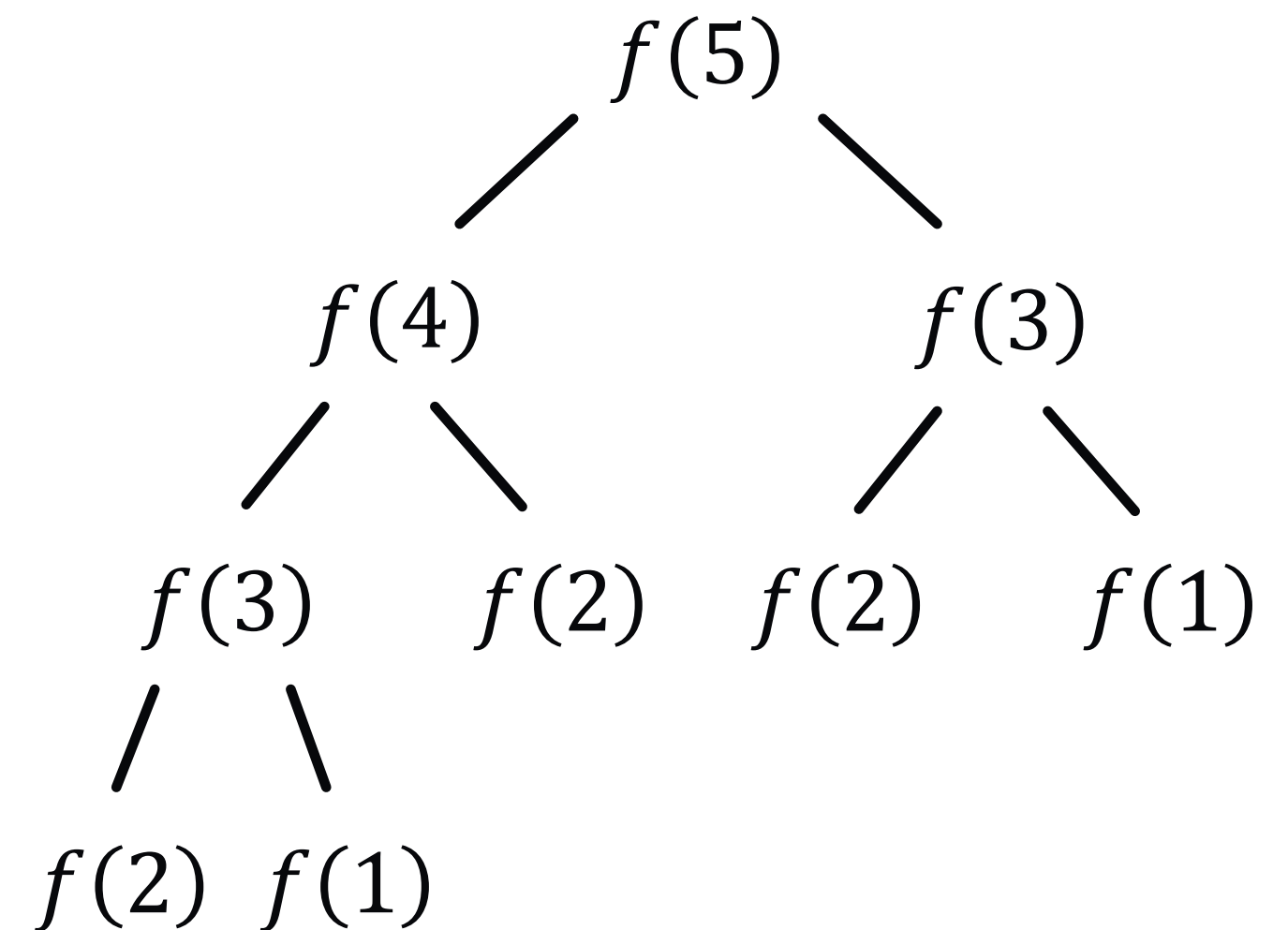
하위 문제가 서로 독립적이며 중복되지 않는다

하위 문제가 중복되므로 결과를 저장하여 재사용한다

분할정복법



동적계획법





시간 복잡도 계산하기

간단한 재귀호출

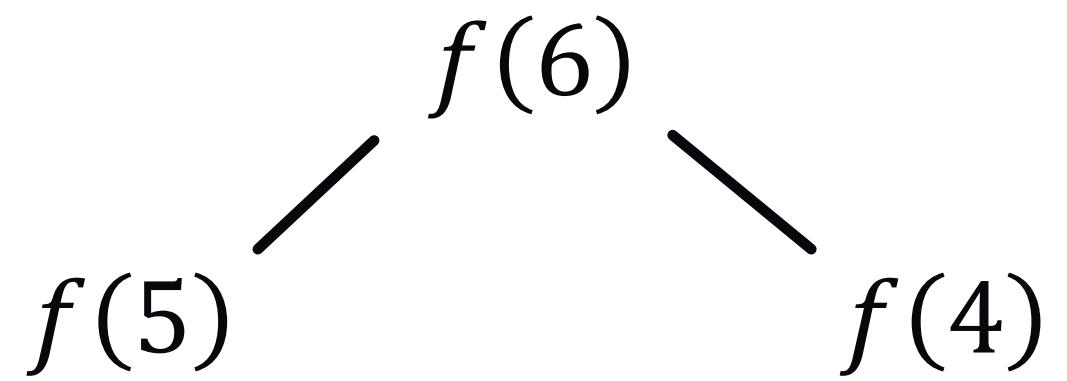
동적계획법

$$f(6)$$



시간 복잡도 계산하기

간단한 재귀호출



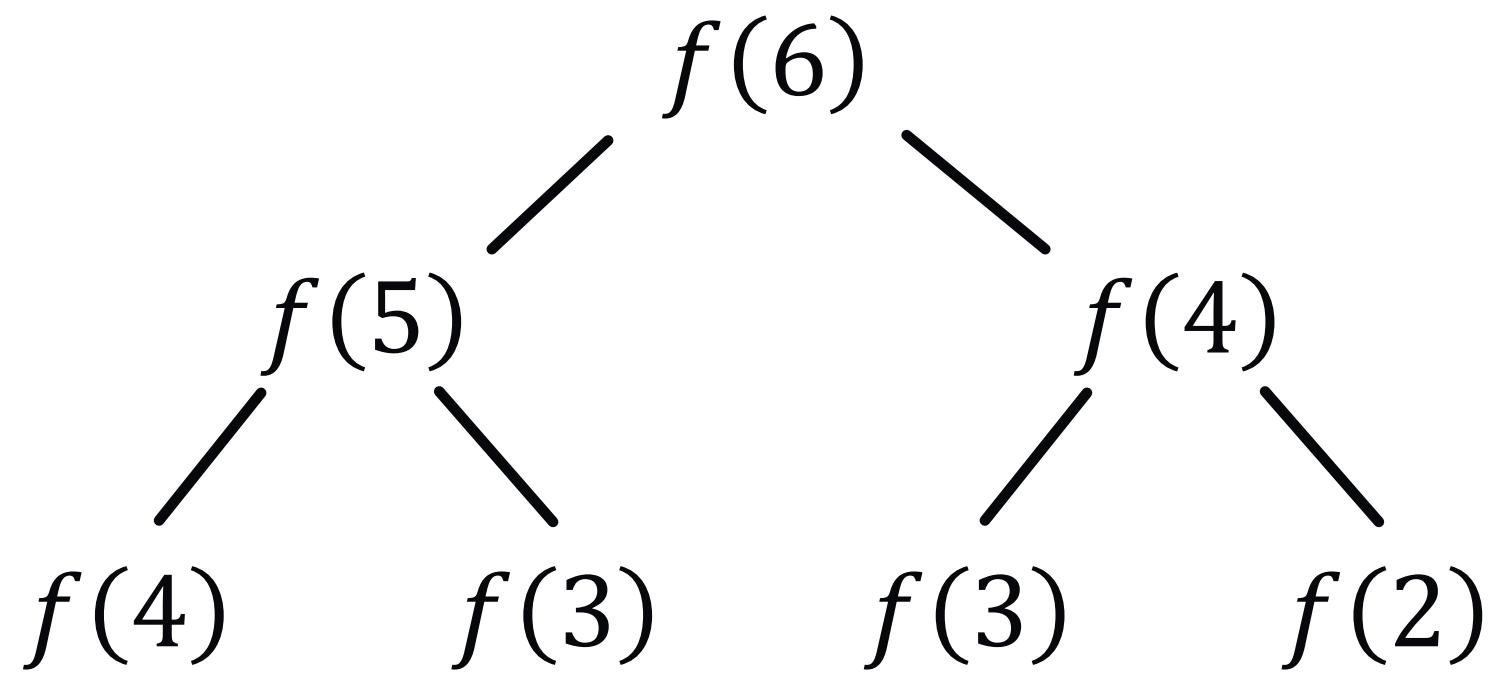
동적계획법



시간 복잡도 계산하기

간단한 재귀호출

동적계획법

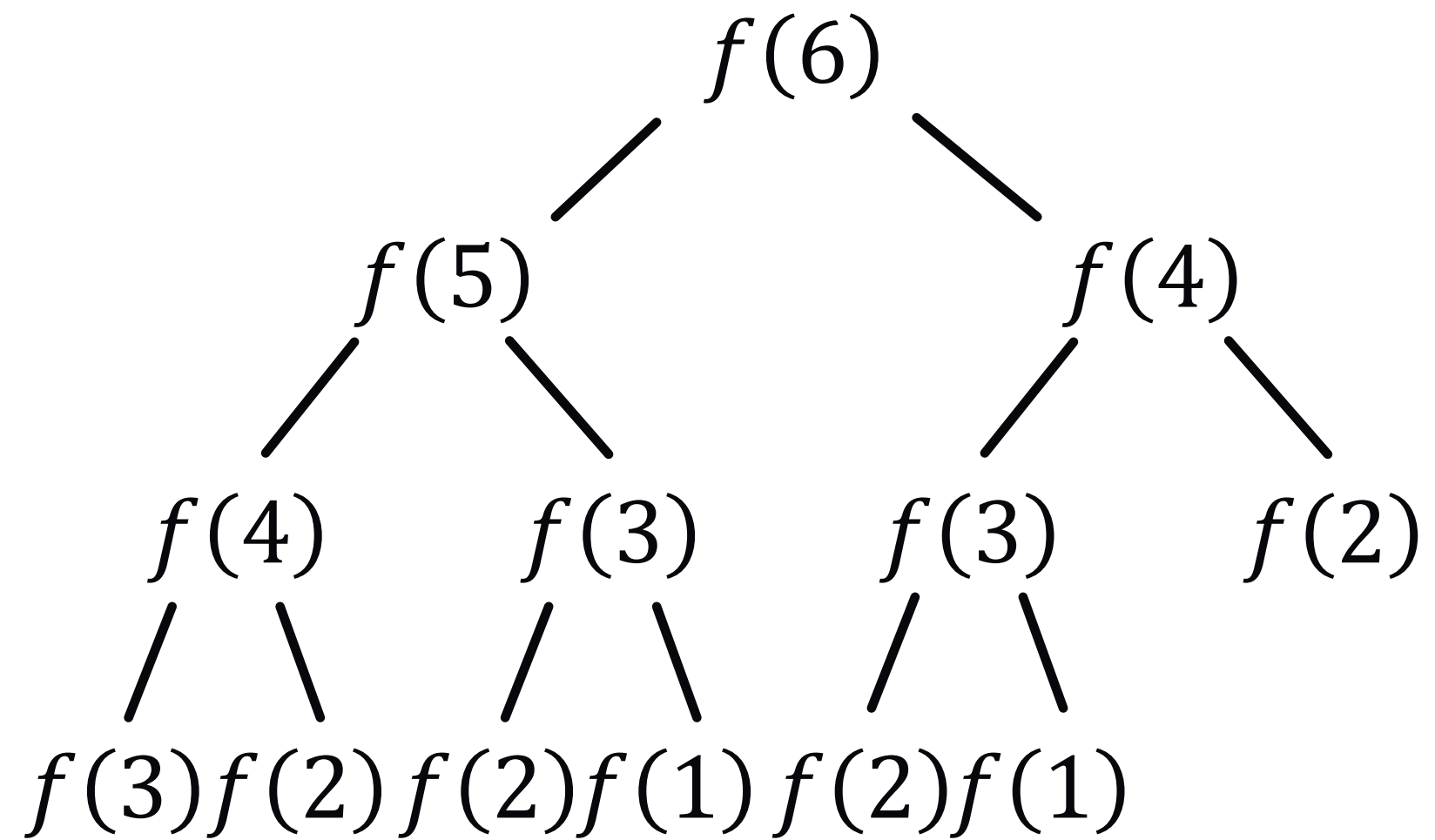




시간 복잡도 계산하기

간단한 재귀호출

동적계획법

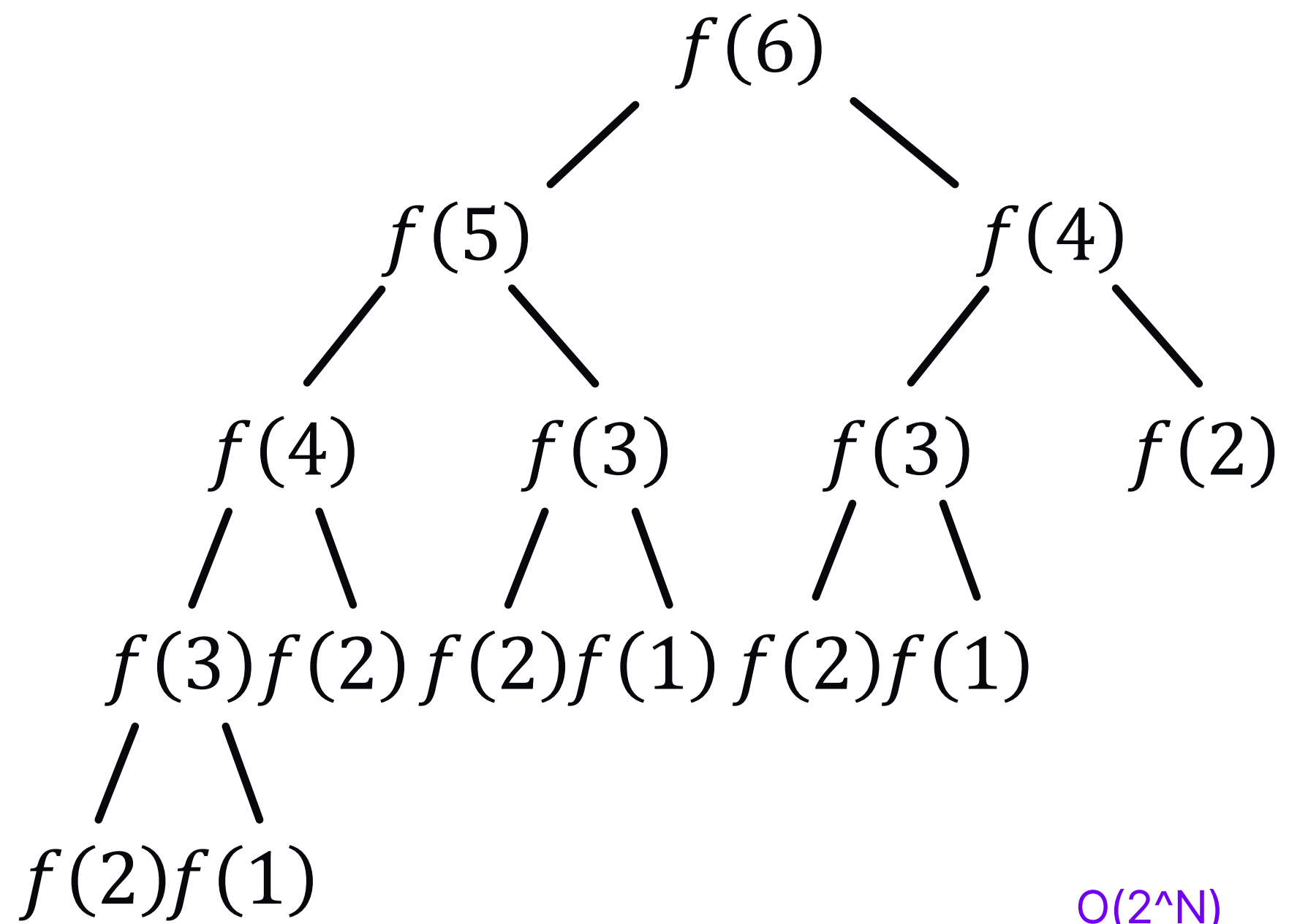




시간 복잡도 계산하기

간단한 재귀호출

동적계획법

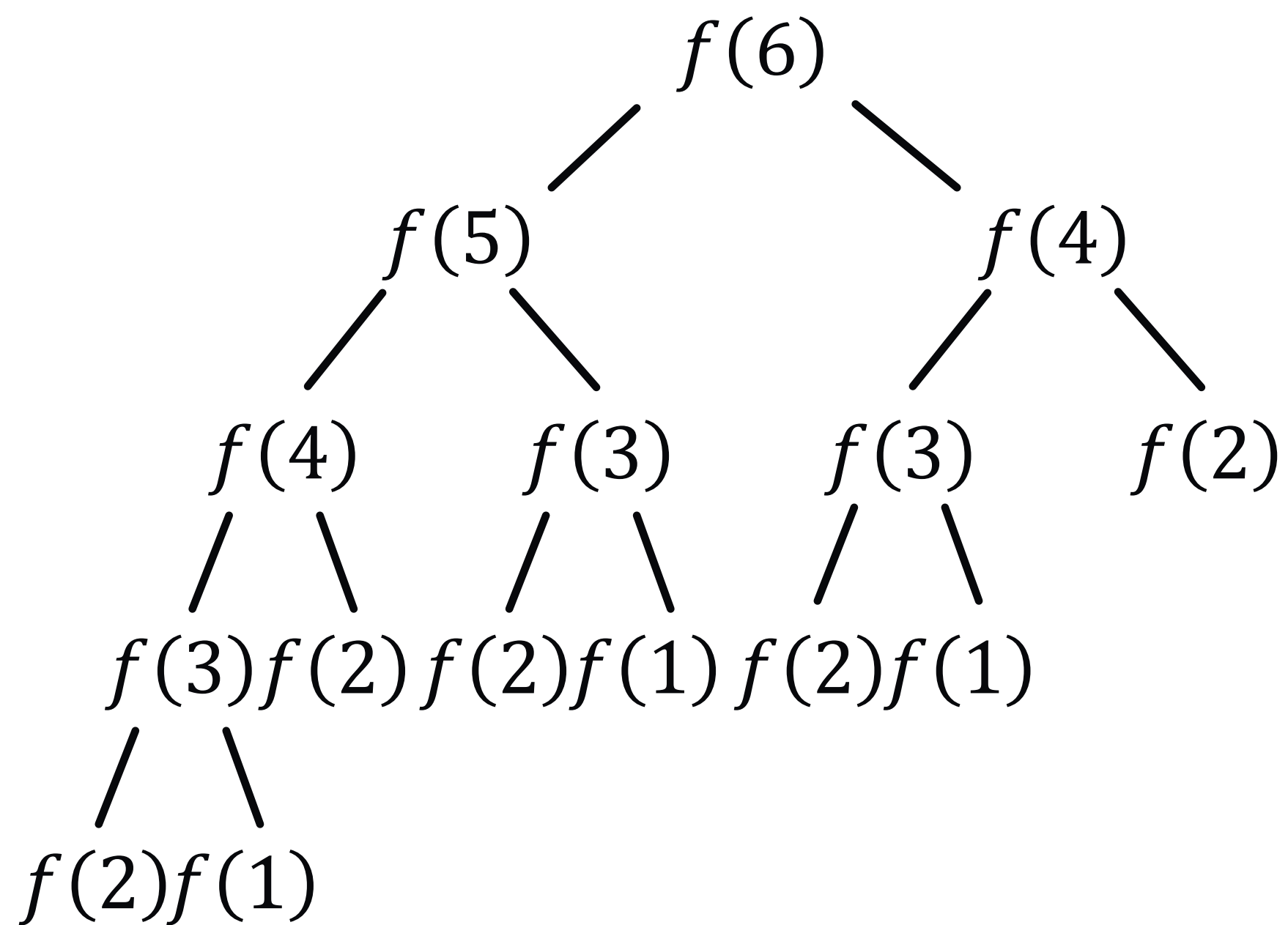


$O(2^N)$

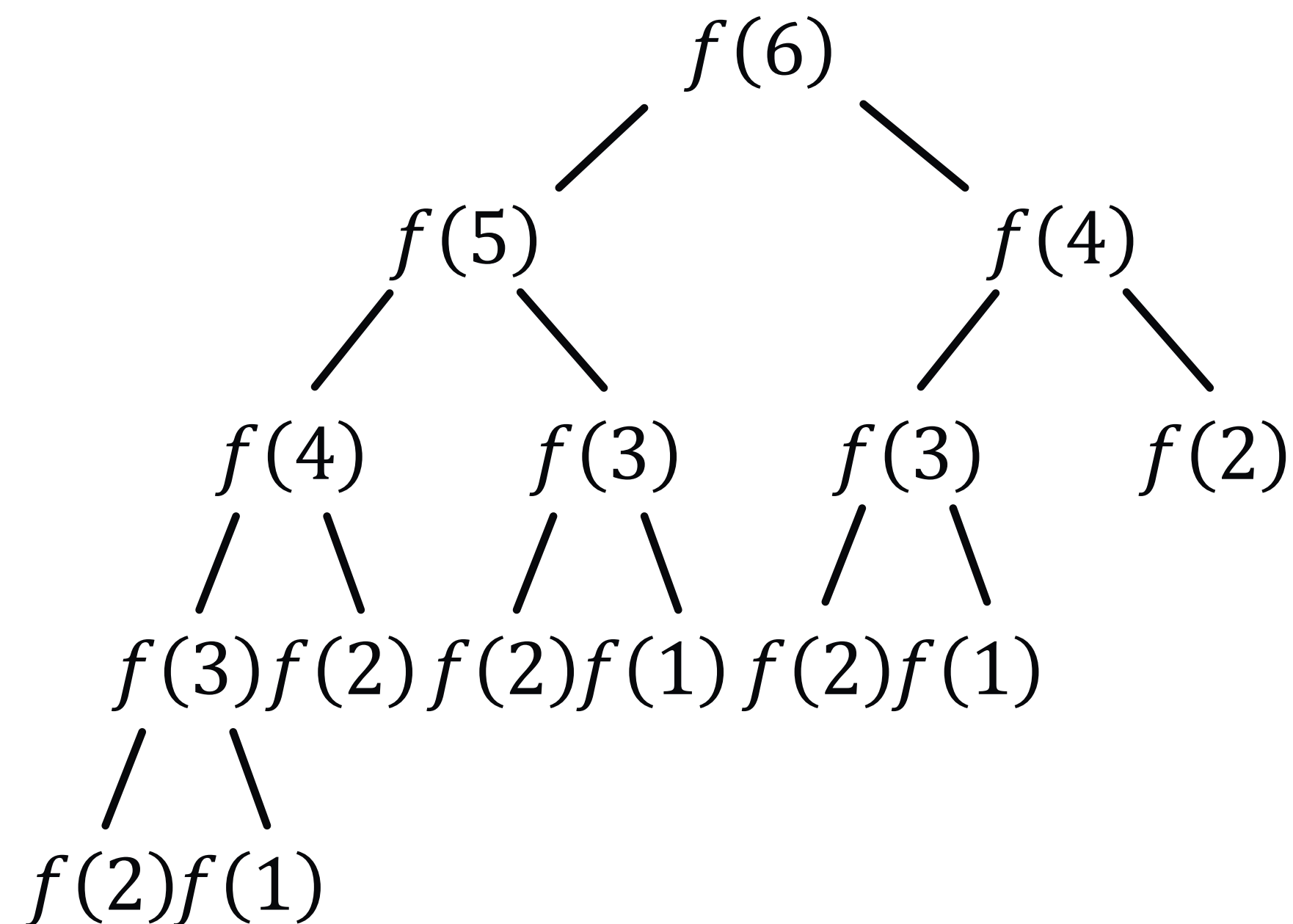


시간 복잡도 계산하기

간단한 재귀호출



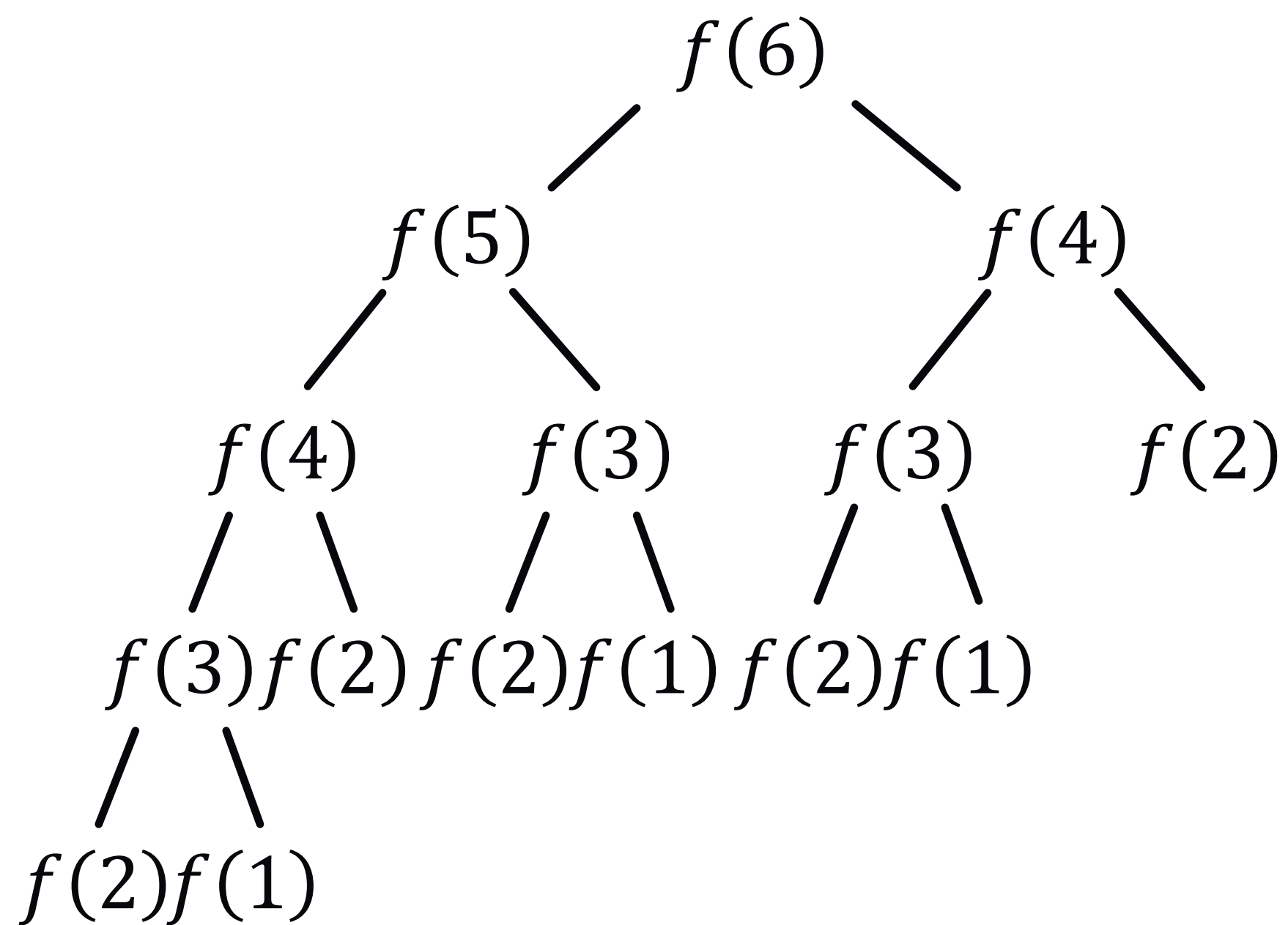
동적계획법



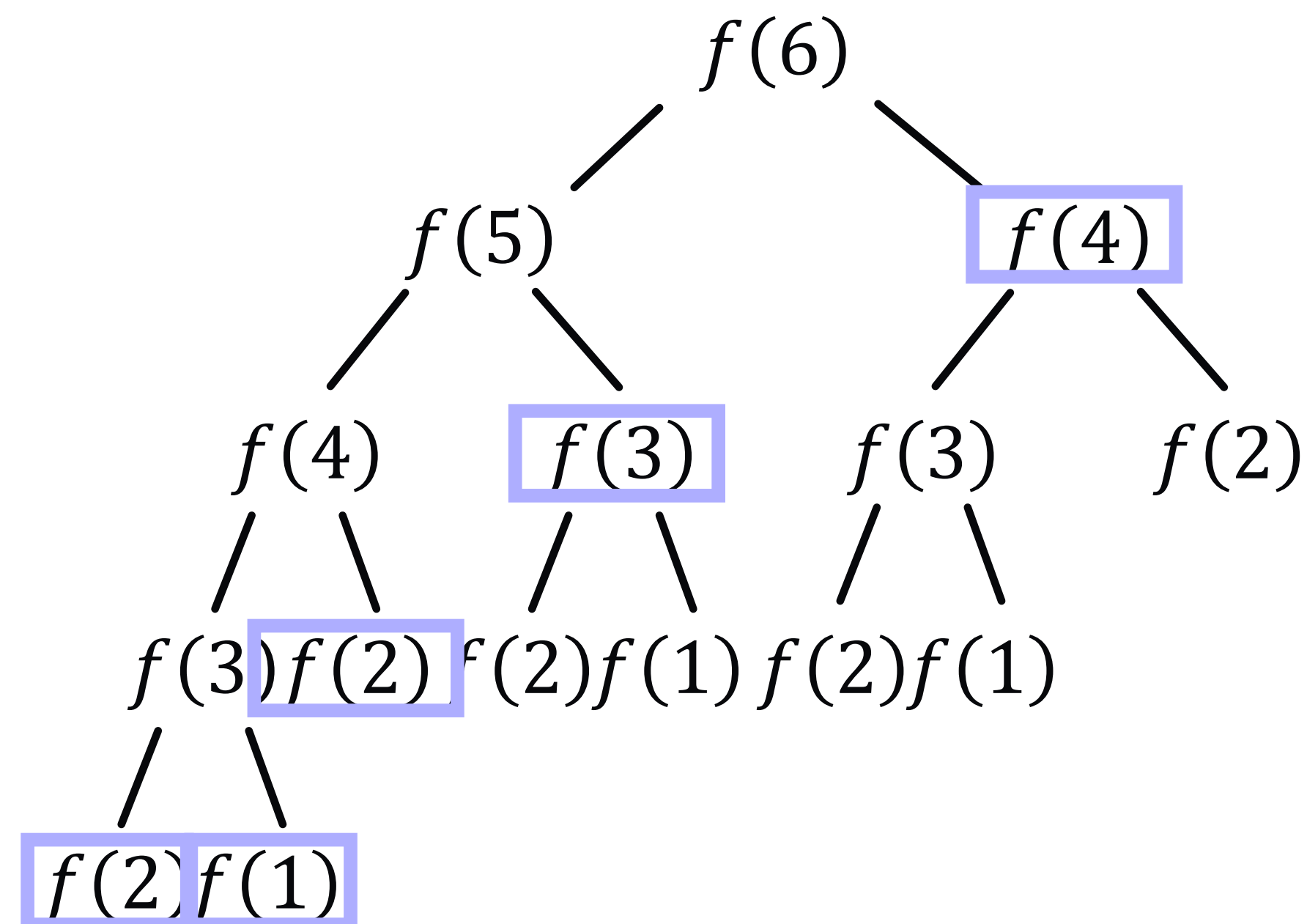


시간 복잡도 계산하기

간단한 재귀호출



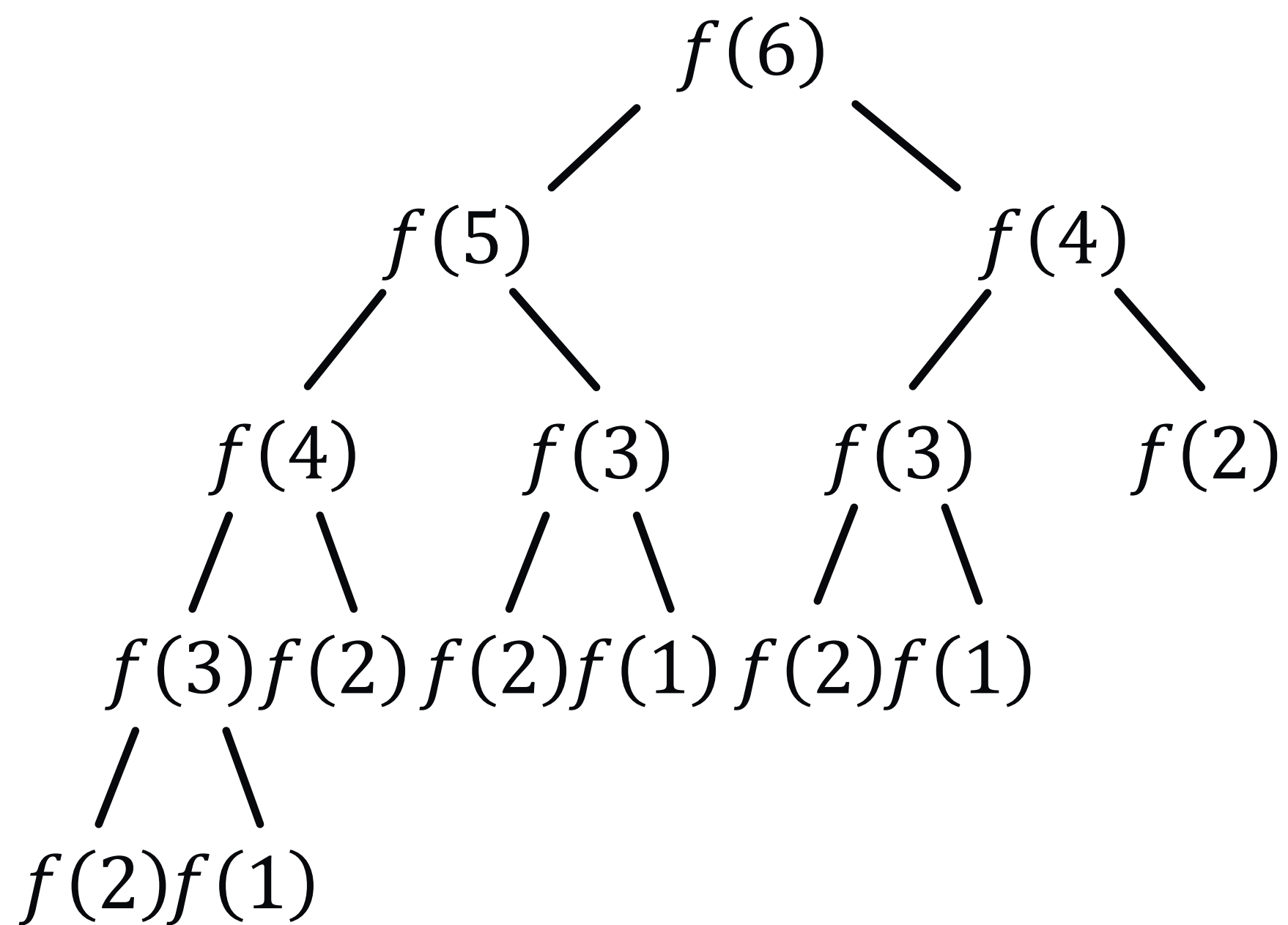
동적계획법



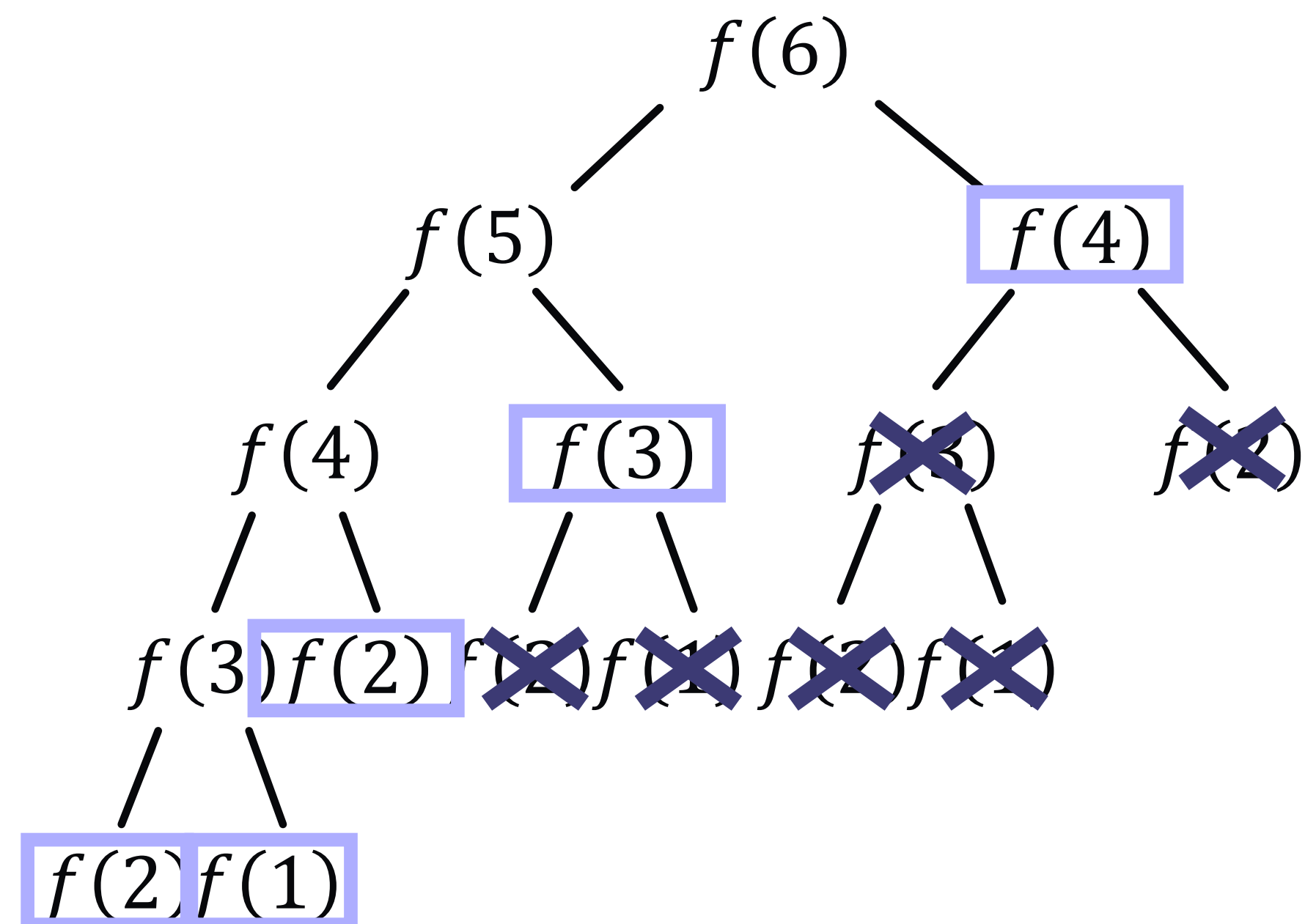


시간 복잡도 계산하기

간단한 재귀호출



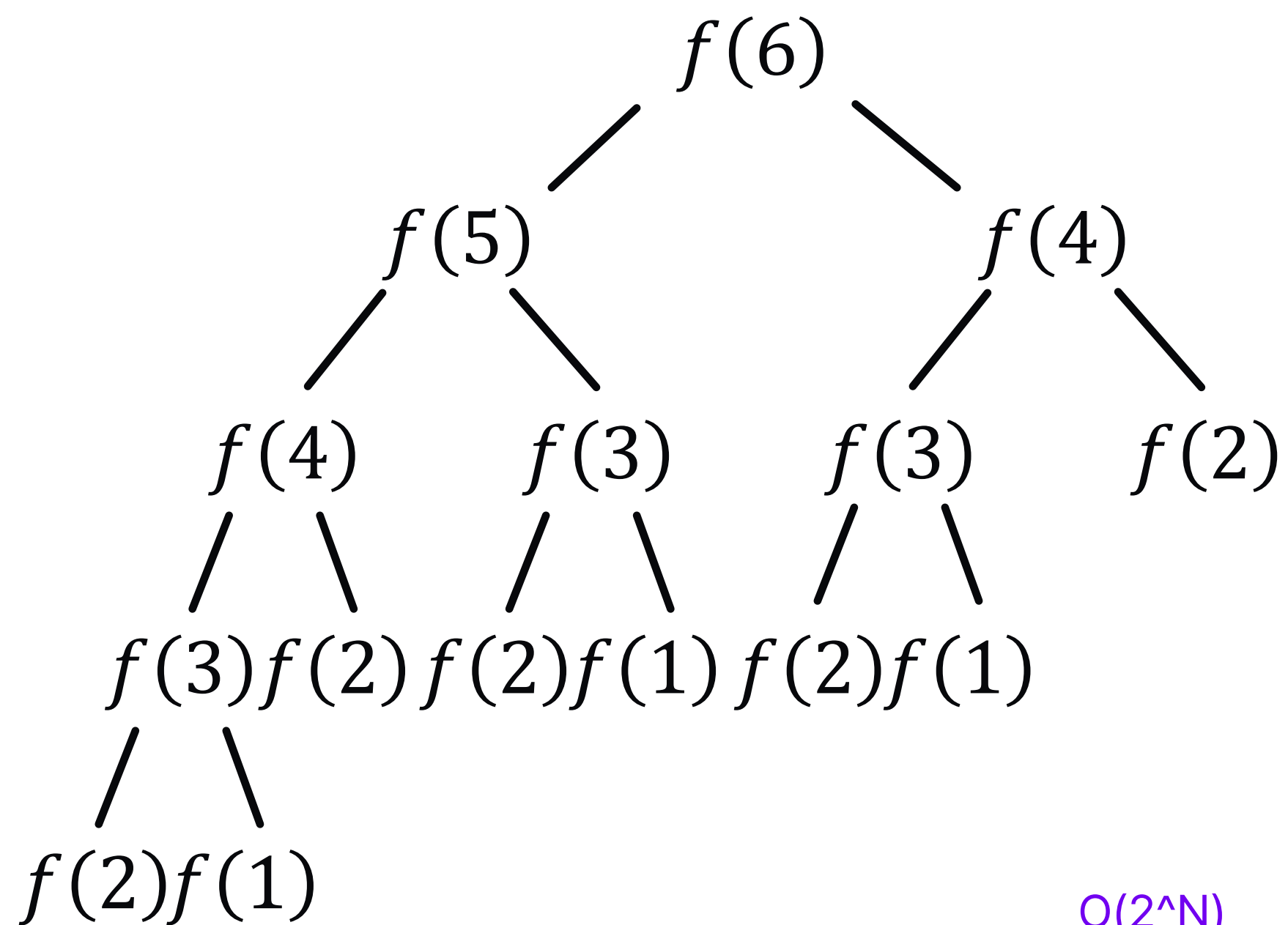
동적계획법



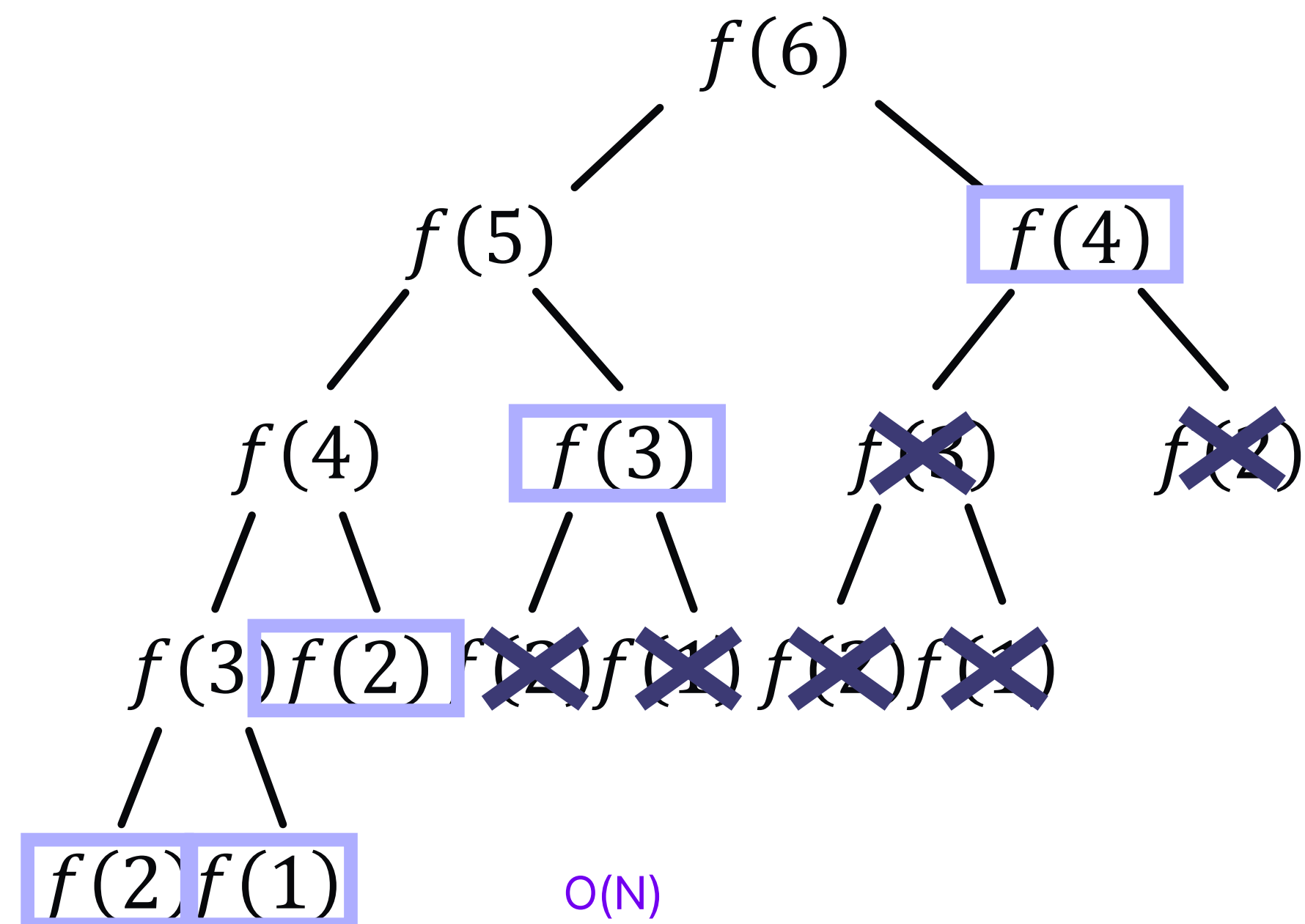


시간 복잡도 계산하기

간단한 재귀호출



동적계획법





시간 복잡도 계산하기

간단한 재귀호출

```
def fibo_1(n):  
    if n < 3:  
        return 1  
    else:  
        return fibo_1(n-1) + fibo_1(n-2)
```

✓ 0.0s

```
%%time  
fibo_1(30)
```

✓ 0.2s

CPU times: user 208 ms, sys: 5.14 ms, total: 213 ms
Wall time: 217 ms

832040

동적계획법

```
fibonacci = {1: 1, 2: 1}  
  
def fibo_2(n):  
    if n in fibonacci:  
        return fibonacci[n]  
    else:  
        fibonacci[n] = fibo_2(n-1) + fibo_2(n-2)  
        return fibonacci[n]
```

✓ 0.0s

```
%%time  
fibo_2(30)
```

✓ 0.0s

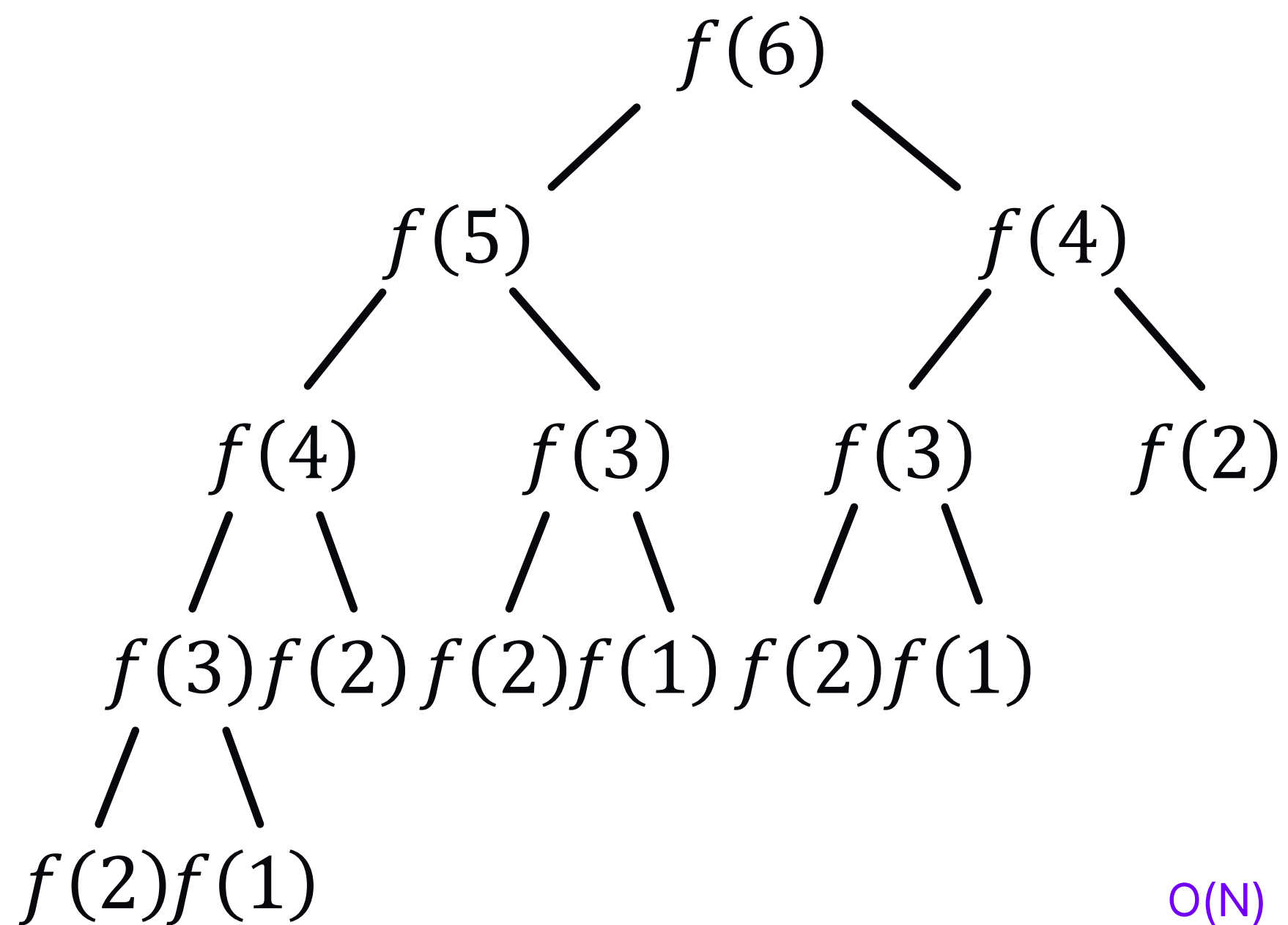
CPU times: user 78 µs, sys: 10 µs, total: 88 µs
Wall time: 191 µs

832040



공간 복잡도 계산하기

간단한 재귀호출



$O(N)$

재귀호출은

함수 호출 스택이 N 만큼 쌓임

동적계획법은 하위 문제들의 답을 저장
→ 하위 문제의 수만큼 저장공간이 필요

 $O(N)$



점화식

복잡한 문제를 작은 하위문제로 표현한 식

예) $f(n) = f(n-1) + f(n-2)$

1. **구하고자 하는 값**이 무엇인지 정의한다

$f(n)$: n 번째 피보나치 수열

2. 구하고자 하는 값을 **부분문제들로 표현**한다

피보나치 수열의 n 번째 항은 $n-1$ 항과 $n-2$ 항의 합이다.

$$f(n) = f(n-1) + f(n-2)$$



점화식

복잡한 문제를 작은 하위문제로 표현한 식

예) $f(n) = f(n-1) + f(n-2)$

Top-down

1. 큰 문제를 작은 문제로 나눈다
2. 작은 문제를 풀어 return해준다

Bottom-up

1. 작은 문제부터 차례로 풀어 적는다.
2. 크기를 조금씩 늘려서 문제를 푼다



Top-down

```
fibonacci = {1: 1, 2: 1}

def fibo(n):
    if n in fibonacci:
        return fibonacci[n]
    else:
        fibonacci[n] = fibo(n-1) + fibo(n-2)
    return fibonacci[n]
```

Bottom-up

```
def fibo(n):
    fibonacci = [-1, 1, 1]
    if n < 3: return 1
    for i in range(3, n+1):
        fibonacci.append(
            fibonacci[i-1] + fibonacci[i-2])
    return fibonacci[n]
```




Top-down

- 가장 큰 문제부터 풀기 시작하여, 작은 문제들을 재귀적으로 호출하여 답을 구하는 방식
- 주로 재귀를 통해 해결하고, 이때 메모이제이션(Memoization)을 활용하여 복잡도를 줄임

```
fibonacci = {1: 1, 2: 1}

def fibo(n):
    if n in fibonacci:
        return fibonacci[n]
    else:
        fibonacci[n] = fibo(n-1) + fibo(n-2)
    return fibonacci[n]
```

Bottom-up

- 작은 문제들을 먼저 풀기 시작하여, 최종적으로 가장 큰 문제들을 해결하는 방식
- 주로 반복문을 통해 해결하고, 점화식과 기저 사례(base case)를 기반으로 결과를 테이블에 저장(Tabulation)

```
def fibo(n):
    fibonacci = [-1, 1, 1]
    if n < 3: return 1
    for i in range(3, n+1):
        fibonacci.append(
            fibonacci[i-1] + fibonacci[i-2])
    return fibonacci[n]
```




동적계획법을 사용하는 조건

- 두 가지 속성을 만족해야 동적 계획법으로 문제를 해결할 수 있음
- **겹치는 부분(작은) 문제**(Overlapping Subproblem)
 - 어떠한 문제가 여러 개의 부분(하위) 문제(subproblem)으로 쪼갤 수 있을 때 사용
- **최적 부분 구조**(Optimal Substructure)
 - 문제의 정답을 작은 문제의 정답에서 구할 수 있을 때 사용
- 예시 : N 번째 피보나치 수를 구하는 문제
 - $N - 1$ 번째 피보나치 수를 구하는 문제, $N - 2$ 번째 피보나치 수를 구하는 문제(하위 문제)로 쪼갤 수 있음
 - 문제의 정답을 하위 문제의 정답을 합하는 것으로 구할 수 있음